Building the simulated Night Ride evening

Scenario methodology and simulator contract, explained

An explanatory companion to the scenario methodology (v1.6), the scenario specification (v0.8) and the model design (v0.5)

Status	Companion for methodology v1.6. Section 7: verified M0–M2 and the M3 batch.
Date	23 October 2026
Sources read	Versions and fingerprints in Section D

What this document is, and is not

This is a **reading companion**. It explains how one simulated Night Ride evening is built from April 2026's aggregate dashboard, what the simulator must do, and how the van dispatcher scores its plans. It introduces **no new rule, number or decision**. Every number is quoted from the documents listed in Section D; where they disagree, the decision log wins, then the scenario specification. Points where the sources disagreed while this companion was written, and how the authoritative documents settled them, are listed in Section C.

Only Section 7 reports simulated outcomes, and only from designed cases: the validation levels L1–L4 and the two-van L5 rehearsal nights. They describe behaviour; nothing here ranks the Lyft rules or says any rule is better. Everywhere else, outcomes are design statements or hand-computed checks.

How to read this document

Sections 1 to 6 are meant to be read in order. Section 1 states the question and the boundaries of the system. Section 2 explains what April's aggregate data can and cannot tell us. Section 3 builds a scenario one block at a time. Section 4 restates the simulator's rules in plain words, Section 5 explains the dispatcher's mathematics, and Section 6 follows one request from start to finish. The appendices hold the decision history, the symbols, the open points and the references.

Three kinds of statement are easy to confuse in a simulation study, so every important statement carries a marker:

Marker	Meaning
ASSUMED	An input rule or behaviour that *we chose*. It is not a measured Hopkins fact. Example: riders' patience follows a placeholder distribution.
DESIGNED CHECK	A small validation case built by hand (levels L1–L4) whose correct answer is known in advance. It checks the machinery, not the research question.
SIMULATED	An outcome that *only the simulation produces*: waits, cancellations, no-shows, transfers, completions. In this document such outcomes appear only in Section 7, from a designed validation case.
APRIL EVIDENCE	A number read from April 2026's aggregate dashboard, as quoted in the source documents.
DERIVED	Arithmetic on April numbers (for example a ratio), resting on stated assumptions.

Decisions also carry an **approval status**, because several 21 October conventions are proposals rather than team approvals:

accepted	The team decided it (or it corrects a demonstrable error).
provisional	A working value or choice that may change, usually after calibration or a benchmark.
proposed, not approved	Written down by the consolidation review of 21 October but not separately approved by the team.
open	No decision yet.

Decision identifiers such as D-36 refer to rows of the project decision log (Section A). File names such as `scenario_spec.md` refer to the repository's `docs/technical/` folder.

Contents

How to read this document **1**

1 The research question and the boundaries of the system **6**

1.1 The question . . . 6
1.2 Three things kept apart . . . 6
1.3 System boundaries . . . 6
1.4 From the September proposal to the current study . . . 8
1.5 How results will be judged (pointer) . . . 9

2 What April's aggregate evidence identifies, and what it leaves unknown **10**

2.1 What the dashboard gives . . . 10
2.2 Why aggregates cannot pin down one scenario . . . 11
2.3 Calibration keeps a set, not a point . . . 11

3 Building a scenario, block by block **12**

3.1 Volume: how many requests . . . 12
3.2 Hourly timing: when within the hour . . . 14
3.3 Geography: where requests start and end . . . 14
3.4 Origin–destination pairing . . . 15
3.5 Parties: how many riders per booking . . . 15
3.6 Fleet . . . 16
3.7 Patience: when a waiting rider gives up . . . 16
3.8 No-shows: who is at the curb . . . 17
3.9 Lyft: what happens after a transfer . . . 18
3.10 Patterns, seeds and pairing . . . 19

4 The simulator contract in plain language **19**

4.1 Time and travel . . . 20
4.2 Request states and commitment . . . 20
4.3 What happens when several events share a second . . . 21
4.4 Epochs: when the dispatcher re-plans . . . 21
4.5 The Lyft rule pass . . . 21

4.6 Transfers and the Lyft service . . . 22
4.7 Shifts and closing . . . 22
4.8 Inputs the simulator must not read . . . 23
4.9 Required logs . . . 23
4.10 Conventions fixed when the simulator was built . . . 24
4.11 Terminal states and the dashboard view . . . 25
4.12 Designed checks: the level ladder . . . 25

5 The dispatcher's mathematics 26
5.1 The idea in words . . . 26
5.2 What a "good" plan means . . . 26
5.3 The route-selection model . . . 28
5.4 Worked example: why the long waiter goes first . . . 29
5.5 What "optimal" means here . . . 31
5.6 The dispatchers and the held-fixed choice . . . 31

6 One request's lifecycle 32
6.1 The van path . . . 32
6.2 The transfer path . . . 34
6.3 Where each hidden trait enters . . . 34

7 Implementation status, provisional assumptions and next milestones 35
7.1 What was built . . . 35
7.2 What was verified . . . 36
7.3 What M1 showed on L4 under R0 (original pending price) . . . 36
7.4 A property of the objective that M1 exposed . . . 37
7.5 The pending-price decision experiment . . . 37
7.6 M2: the Lyft rules with the dispatcher, over the whole L4 evening . . . 38
7.7 M3: the L5 pilot and the 30-night batch . . . 40
7.8 Reading the results as trade-offs . . . 42
7.9 A leader–follower reading (possible extension, not implemented) . . . 43
7.10 Status by item . . . 44
7.11 Limitations to keep in view . . . 44

7.12 Next milestone and how the work grows 45
7.13 Open decisions for the team 46

A Decision history **47**

B Symbols and units **50**

C Points where the sources disagreed, and how they were settled **51**

D Source documents and versions **52**

E References **53**

List of Figures

1 The system in one picture 7
2 Largest-remainder rounding of the hourly request counts 13
3 Three assumed input distributions 17
4 Two denominators for the same no-shows 18
5 Timeline of a transferred rider 23
6 The convex waiting penalty 27
7 Worked example: two candidate routes 30
8 The life of one request 33
9 Lyft use against service on L5 43

List of Tables

1 Inputs, policies and outcomes are kept apart (after the specification §1 and the methodology §3). 6
2 From the proposal to the current study 9
3 April 2026 aggregates used in this document 10
4 Toy example: one set of margins, two trip tables 11
5 The Lyft transfer rules 19
6 Demand patterns (`scenario_spec.md` §3.4). 19
7 Hidden per-request draws 20
8 Same-second event order 21

9 The level ladder . . . 25
10 Worked example arithmetic . . . 30
11 Van dispatchers . . . 32
12 Where each hidden trait turns into an outcome. . . . 34
13 What M0–M1 verified . . . 36
14 The 18:15 plan in L4 under R0 . . . 37
15 Pending-price formulations compared on L4 under R0 . . . 38
16 M2 on L4: one designed integration evening . . . 39
17 Planned drop-offs before and after the 18:20 re-solve under R2(15) . . . 39
18 What happened to transferred riders on L5 . . . 41
19 Descriptive measures on L5 . . . 42
20 Status by item after M0–M3's batch (`scenario_methodology.md` v1.6 §7). . . . 44
21 Milestones (`scenario_methodology.md` v1.6 §8). M0–M2 and the M3 batch are done; the rest is a plan. . . . 45
22 Items that are provisional, proposed or open . . . 46
23 Decision history . . . 47
24 Symbols and units. . . . 50
25 Disagreements found in the sources and where they were settled. . . . 51
26 Source documents. . . . 52

1 The research question and the boundaries of the system

This section states what the study asks, what is held fixed, and where the modelled system ends.

1.1 The question

Johns Hopkins' Blue Jay Night Ride is an on-demand, shared van service. When the vans fall behind, a human dispatcher can send a waiting rider a Lyft instead. The study asks:

> Holding the van dispatcher fixed, which dispatcher-initiated Lyft transfer rule gives Homewood riders the best outcomes for the resources it uses (Lyft bookings and van-kilometres)?

The wording follows the team's primary question (D-26), refined to *resource use* rather than cost because no money is modelled (D-45). The study is about **policy**, the Lyft rule, not about which algorithm is cleverest. Every rule is run on the same simulated evenings, one rule at a time, with everything else held fixed (D-28).

1.2 Three things kept apart

A fair comparison needs a clean separation between what goes in, what we vary, and what comes out (Table 1). Inputs are identical for every rule; the rule is the only thing that changes; outcomes are produced only by the simulation, never typed in.

Table 1: Inputs, policies and outcomes are kept apart (after the specification §1 and the methodology §3).

Kind	Examples	Who sees it
Inputs (public)	Request time, origin, destination, party size; vans on duty	The dispatcher, but only once a request has arrived
Inputs (hidden)	Patience deadline, presence draw, consent draw, Lyft delay	Only the simulator, and only when the relevant event happens
Policy	Dispatcher, objective weights, re-planning cadence, Lyft rule, dispatcher lag	Fixed for a run
Outcomes SIMULATED	Actual waits, pickups, cancellations, no-shows, transfers, Lyft completions, van-km	Evaluation only

1.3 System boundaries

The boundaries below are fixed for the main experiments. Each has a decision behind it. Figure 1 shows how the parts fit together.

- **Area:** Homewood Night Ride *including the Peabody area*, as April was operated (D-27) accepted. East Baltimore demand, policies and targets are excluded (D-23) accepted.

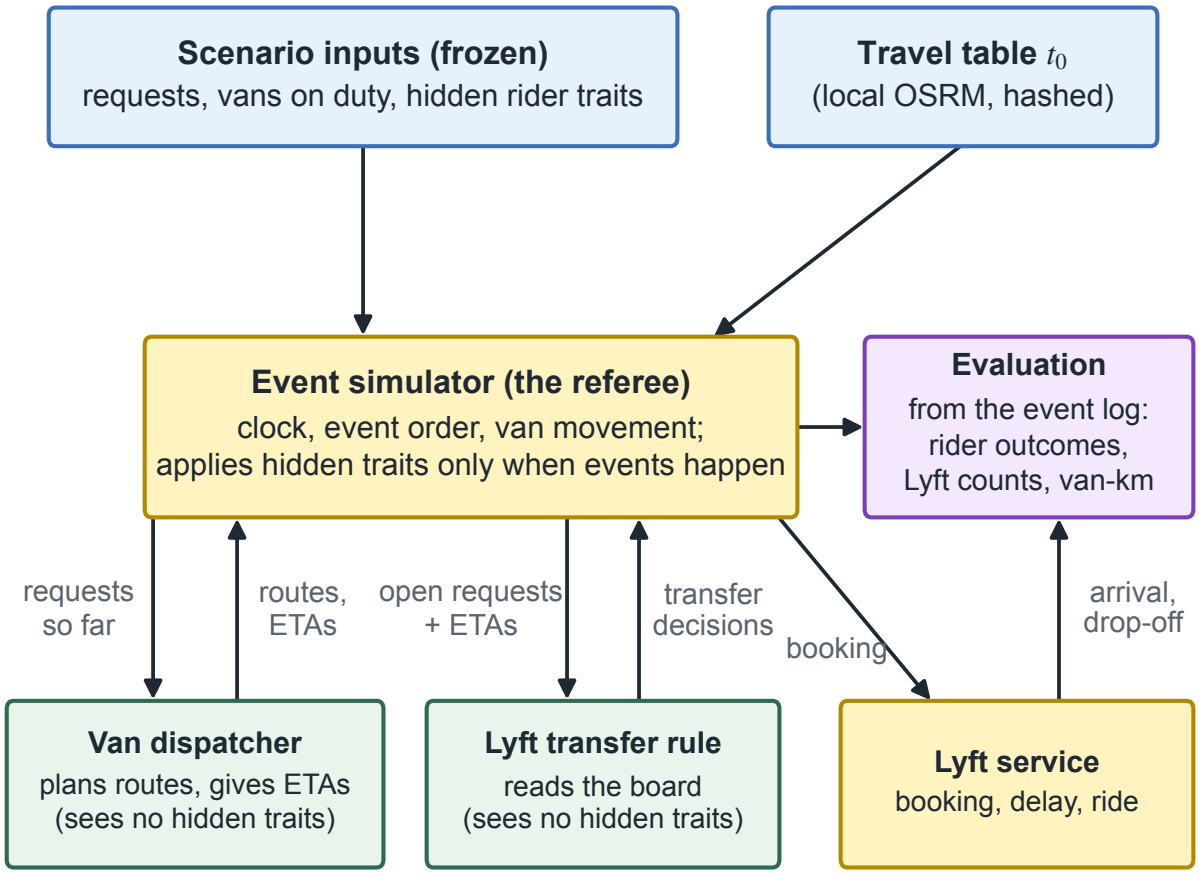


Figure 1: The system in one picture. The frozen scenario and the travel table feed the event simulator, which acts as referee: it owns the clock and applies each rider's hidden traits only when the relevant event happens. The van dispatcher and the Lyft transfer rule (green) are policies; they see only what a real dispatcher could see. The Lyft service is deliberately thin: the study needs *when* a transferred rider is picked up, not how Lyft matches drivers. After `scenario_methodology.md` §2.

- **Door-to-door service, no stop experiment:** riders are picked up at a snapped curb point near their location. The digitized April hotspots are *demand-generation points*, not stops; stop consolidation is out of scope (D-24) accepted.
- **When requests arrive:** 18:00–21:00, the busiest three hours, beginning at the first hour of service so no backlog is inherited (design trail §1) accepted. After 21:00 no new requests arrive, and the run continues until every accepted request is resolved (the closing rule, D-36; build convention D-46(6) proposed, not approved).
- **Fleet:** five vans with 14 seats each, on duty throughout 18:00–21:00 and then until the backlog is finished (D-36) accepted. The benefit of a sixth van is measured on the *same* requests (D-37) accepted.
- **Parties of five:** a party of five that is transferred travels together as *one* Lyft XL booking; parties of one to four use a standard Lyft (D-33). This is an accepted modelling instruction based on user-supplied operational knowledge, not verified Hopkins policy.
- **No money:** the model has no fares, fees or dollar expenditure, and unknown prices are not set to zero. Resource use is reported as counts: transfer decisions, bookings, completed Lyft trips (standard and XL separately), van-km (D-45) accepted.
- **Service quality is not safety:** shorter outdoor waits at night are a service-quality proxy for the program's safety purpose. The model measures waits, cancellations and ride times, not physical safety (`model_design.md` §4.4).
- **Targets as reporting thresholds:** 15-minute pickup, 20-minute ride and 20-minute busy-time arrival, taken from the JHU page wording captured around April 2026. The live page read on 21 October 2026 states 20/25/25. The two readings are kept distinct, and neither is treated as proof of operating policy (D-49).

1.4 From the September proposal to the current study

The September proposal planned to replay real service nights from trip-level records and to compare dispatch methods head to head: greedy rules, a rolling-horizon MILP and a hybrid. It put Lyft inside the optimizer as paid overflow and treated the published targets as hard constraints. Each of these changed, for a stated reason (Table 2; `scenario_methodology.md` v1.6 §1a).

Table 2: From the September proposal to the current study.

Proposal	Current study	Why
Trip-level records "coming"	Only aggregate dashboards (totals, shares, percentiles)	The records did not arrive; every scenario is synthetic and built from aggregates (Section 2)
Compare dispatch algorithms	Compare Lyft-transfer rules with the van dispatcher held fixed	A scope choice: we specify one van dispatcher, modelled on TransLoc's published 2016 design (D-25, D-51), and hold it fixed to isolate the effect of the transfer rule. Synthetic nights do not rule out algorithm comparisons; whether Hopkins' current dispatcher works this way is not verified
Lyft as paid overflow inside the MILP	A separate, explicit transfer rule; no money modelled	The JHU page says a dispatcher sends the Lyft; Hopkins' cost per Lyft trip is unknown (D-45)
Targets as hard constraints	Reporting thresholds, plus a ride guardrail	Hard targets would be stricter than the observed service, and the targets' wording changed over time (D-47, D-49)
Claims about real nights	Claims about synthetic evenings consistent with April's totals, under stated assumptions	Section 2

Proposal numbers, corrected APRIL EVIDENCE**:**

- "About 5,100 rides per hour at 6 pm" is the *30-day total* for the 6 pm clock hour (5,098 on the dashboard's tooltip, 5,089 digitized), that is, about 170 completed rides per evening.
- "About 28 vans" is the "All services" average (27.6, over 29 days), not a Homewood-only count.
- The 20/25-minute targets come from the live page read on 21 October 2026; the pages captured around April 2026 stated 15/20/20 (D-49).
- "1 in 4 trips overflow to Lyft" is a share of *riders* (24.5% of 44,374 riders, van and Lyft together), not of trips or requests.

The proposal deck itself is kept unchanged as a record.

1.5 How results will be judged (pointer)

The rules for judging results live in the draft pre-registration (`preregistration_v2.md`), which is frozen when the first main (L6) comparison run starts. In outline:

- Rules are compared on **paired** nights (the same 30 evenings for every rule), and the answer is a **frontier** of rider outcome against resource use, not a single winner. "Send everyone to Lyft" is one end of that frontier, never the answer.
- The proposed primary rider outcome is the share of *all* requests not reached by a vehicle (van or Lyft) within 15 minutes, counting riders who cancelled as not reached (D-50(2) proposed, not approved). Wait percentiles among picked-up riders leave out those who gave up, so they are read beside it.

- Comparisons use non-inferiority margins rather than "the confidence interval contains zero". The correction of that wording is accepted; the margins themselves (1 minute for wait percentiles, 2 percentage points for shares) are proposed, not approved (D-50).

2 What April's aggregate evidence identifies, and what it leaves unknown

This section separates what the April data can support from what has to be assumed. It explains why the study builds a family of plausible evenings rather than one reconstructed night.

2.1 What the dashboard gives

The only operational source is the April 2026 TransLoc dashboard for the Homewood Night Ride. It is restricted, so this document uses only the aggregate numbers already quoted in the project's documents. It gives **totals and shares**: requests by status, completed rides by hour, average vehicles by hour, origin and destination density maps with printed bubble counts, and a few wait and ride percentiles. Table 3 lists the numbers this document relies on.

Table 3: April 2026 aggregates used in this document, as quoted in the sources. All are APRIL EVIDENCE except the last block, which is DERIVED under stated assumptions.

Quantity	Value	Where quoted
Recorded requests (Completed + Canceled + No Show; Denied read as 0)	50,297	spec §3.2
Completed rides	24,095	spec §3.2
"No Show"	2,359 (4.7% of all requests; 2.8–6.4% by day)	trail §8
"Canceled"	≈ 47% of requests (31–68% by day)	spec §12
Passengers per completed booking	1.386	corrections, 21 Oct
Lyft share of riders	≈ 24.5% (10–46% by day)	spec §12
Completed-ride wait	median ~10–13 min; P90 ~17–21 min	spec §12
Ride time P90	~24 min	spec §12; model §4.3
Completed rides per van-hour	≈ 5.1 (6.2 at 6 pm)	spec §12
"All services" vehicles at 6 pm	≈ 27.6, tapering later; averaged over 29 days	methodology §4; D-49
Requests per completed ride R/C	50,297 / 24,095 = 2.087 DERIVED	methodology App. B
Load per van ρ_h, 18/19/20 h	12.85 / 10.10 / 10.71 DERIVED	spec §3.2

2.2 Why aggregates cannot pin down one scenario

The dashboard never shows one rider's request, wait and outcome together. That has three consequences, each of which shapes the design (`scenario_methodology.md` §5).

1. Many different nights produce the same totals. The maps give how many trips start in each zone and how many end in each zone, but not which start goes with which end. A toy example makes the point (Table 4): two very different trip tables have exactly the same row and column totals. The same is true of hourly counts, which fit many within-hour patterns. So the design picks one principled member (maximum-entropy pairing; uniform times within the hour) and tests deliberately stressed alternatives (burst, far-trip, dispersed geography). It never claims to be *the* April night.

Table 4: Toy example (not project data). Both tables have origins X = 3, Y = 2 and destinations X = 2, Y = 3, yet they describe different travel patterns.

Table one				Table two			
from \ to	X	Y	total	from \ to	X	Y	total
X	1	2	3	X	0	3	3
Y	1	1	2	Y	2	0	2
total	2	3	5	total	2	3	5

2. Behaviour is confounded with operations. A long wait followed by "Canceled" could be an impatient rider, or a dispatcher who handed the rider to Lyft (which is likely recorded as "Canceled"). The totals cannot separate the two. That is why the rider share of "Canceled" is carried as a 50–75% *range* (D-39), and why patience will be calibrated to a *set* of acceptable values rather than one.

3. Shares do not say who. "4.7% of requests were no-shows" does not say which riders, or whether presence depends on how long they waited. A per-arrival no-show probability follows from the totals only under a stated mapping of dashboard statuses (Section 3.8).

What the study can and cannot claim

The claim is conditional: *in scenarios consistent with April's aggregates, rule X does better than rule Y, and this holds across the identified set and the stress patterns.* The study cannot claim that rule X would have done better on any particular April night.

2.3 Calibration keeps a set, not a point

Before any comparison, the base pattern of the full-night level is to be calibrated against dashboard-comparable bands: completed share ≈ 48%, "Canceled" ≈ 47%, Lyft rider share ≈ 24.5%, no-shows ≈ 4.7% of all requests, completed-ride wait and ride percentiles, and completed rides per van-hour (spec §12). The knobs are the load ratio ρ, the patience scale, the travel multiplier and the threshold of the status-quo rule. Every knob setting that passes is

kept (the *identified set*), and the main comparison is re-run at its extremes (pre-registration §5). Known limits remain: waits are survivor-biased, the cancellation link is correlational, vehicle counts mix services, there are no hourly Lyft data, and the scaled-down system is less dense than the real one.

3 Building a scenario, block by block

A **scenario** is one simulated evening: a list of requests (when each arrives, where it starts and ends, how many riders it carries), the vans on duty, and each request's hidden traits. It is the *same* evening for every rule. A scenario contains inputs only; cancellations, no-shows, transfers and completions are SIMULATED and never written into it.

Each ingredient has its own seeded random stream (times, places, party sizes, patience, no-shows, consent, Lyft delays, and so on). Changing one ingredient never reshuffles the others, and every rule sees identical riders with identical hidden traits. This is the method of *common random numbers*. Each recipe is turned into 30 nights with 30 seeds (spec §0, §11).

Each block below follows the same pattern: the evidence, the calculation or assumption, the chosen input, its limitation, and the sensitivity that tests it.

3.1 Volume: how many requests

The data give the workload per vehicle in April, not the demand a five-van system would face. The design therefore copies April's *load per van*, hour by hour, and gives the scenario's vans that load (D-35).

For each clock hour h:

$$\rho_h = \frac{C_h}{30} \times \frac{R}{C} \div V_h, \qquad N_h = \rho_h \, v_h, \tag{1}$$

where C_h is April's completed rides in that hour over the month, $R/C = 2.087$ is recorded requests per completed ride, V_h is the average "All services" vehicles in that hour, and v_h is the scenario's vans in that hour.

Worked example: 168 requests. With five vans every hour, $5\rho_h$ gives 64.25, 50.50 and 53.55, which sum to 168.3. The *largest-remainder rule* then makes whole numbers that add up to the rounded total:

1. round the total: 168;
2. floor each hour: 64, 50, 53 (sum 167);
3. give the one leftover unit to the hour with the largest fraction (0.55 for 20:00), so the result is **64 / 50 / 54 = 168**.

The same rule with April-shaped availability (5, 4.95, 4.46 vans) gives 64 / 50 / 48 = 162; that tapering case is kept for later.

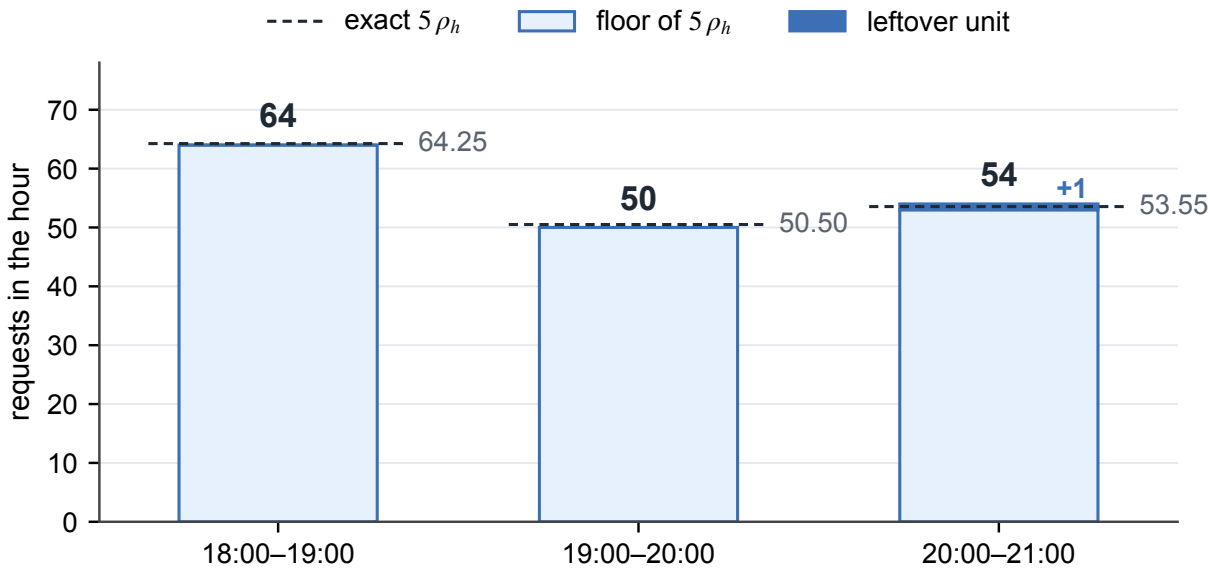


Figure 2: Largest-remainder rounding of the hourly request counts at five constant vans. Dashed lines are the exact values $5\rho_h$; bars are the whole numbers used. The single leftover unit goes to 20:00–21:00, whose fraction (0.55) is the largest.

Evidence	April status totals (requests 50,297; completed 24,095), completed rides by hour, average vehicles by hour ("All services") APRIL EVIDENCE.
Calculation	ρ_h = 12.85 / 10.10 / 10.71 DERIVED; $N_h = \rho_h \upsilon_h$ with largest-remainder rounding. Assumes every status is one request with Denied zero, the same request-to-completion ratio in every hour, completed rides counted by request hour, and "All services" standing in for Homewood vans ASSUMED.
Chosen input	64 / 50 / 54 = 168 requests for 18:00–21:00 at five constant vans (D-36). The availability choice is accepted; the count is provisional (a derived workload, not measured April demand).
Limitation	The vehicle count mixes services and averages 29 days, not 30 (±3% on ρ). Scaling demand down with the fleet keeps load per van but not density, so pooling chances are lower than in the real system: results hold "at the scaled system's density".
Sensitivity	Illustrative range ~150–260 (D-49), *not* a confidence interval: if Homewood had 20–35% fewer vehicles than "All services" the count rises to ≈210–260; ~10% rebookings would lower it to ≈150. The high-load pattern multiplies ρ by 1.3; ρ is the calibration knob.

3.2 Hourly timing: when within the hour

Evidence	Only hourly totals; within-hour bunching is not observed APRIL EVIDENCE.
Assumption	A fixed count per hour at uniformly random times, which is a Poisson process conditional on its count ASSUMED. The hourly chart is read as request hour (D-49(5), a transparent base assumption).
Chosen input	A named "times" stream per level, pattern and night; every rule replays identical times accepted.
Limitation	Night-to-night variation in volume is removed (April's daily requests varied by about 11%). If cancellations concentrate at peaks, the true request profile may tilt earlier.
Sensitivity	Burst pattern: 40% of the 18:00–19:00 requests fall inside one 10-minute block. A random-count (Poisson) variant.

3.3 Geography: where requests start and end

Evidence	Density maps with printed bubble counts, digitized and georeferenced into 188 hotspot points and 16 zones; the city maps sum to 24,056 origins and 24,057 destinations, matching completed rides (D-01) APRIL EVIDENCE.
Assumption	Each hotspot stands for nearby addresses and is used as a demand-generation point; vans stop at the snapped road point (one documented exception, 95 m) ASSUMED. Requests share the geography of completed rides, since cancelled requests' locations are unobserved.
Chosen input	Reference geography: 188 points over Homewood + Peabody (D-27). Travel times come from one base table t_0 (local OSRM v6.0.0, Maryland extract, hashed in `MANIFEST.json`, D-31). Van time $= t_0\, m_v$, Lyft time $= t_0\, m_L$, each multiplier applied once; base $m_v = m_L = 1.2$ ASSUMED, calibrated later. Vans start at a depot on the Homewood campus (D-17) ASSUMED.
Limitation	A city-view bubble stands for a whole neighbourhood (the city view is about four times coarser than the campus view), so the reference stacks many requests on one spot and can flatter pooling exactly where long, isolated trips make Lyft attractive.
Sensitivity	Dispersed variant `dispersed_v1` (D-34): each of the 19 coarse city-view clusters is spread over 6 road satellites within ~400 m, with the same times, cluster-level pairs, parties and traits; its travel table covers 302 points. Its internal consistency is verified; that does not validate 400 m as true pickup locations. Which geography is the L6 base is provisional. Radius 250 / 600 m.

3.4 Origin–destination pairing

Evidence	Origin totals and destination totals per place, not pairs APRIL EVIDENCE.
Calculation	Gravity × iterative proportional fitting (IPF). A pair starts with weight $w_o\, w_d\, e^{-\beta_{\text{geo}}\, t_{od}/60}$, with $\beta_{\text{geo}} = 0.1$ per minute of driving; pairs under 60 s of driving are excluded. IPF then rescales until every place's totals equal April's (largest error $\sim 10^{-10}$) (D-18).
Chosen input	Pairs drawn from that table accepted; β_{geo} provisional. β_{geo} is unrelated to the ride weight $\beta = 0.5$ in the dispatcher's objective.
Limitation	The true pairing is *not identified*: every β_{geo} reproduces the margins (Table 4). The same geography is used in every hour.
Sensitivity	$\beta_{\text{geo}} = 0$ / 0.1 / 0.2; a campus-outbound direction variant; the far-trip pattern doubles the probability of trips with an end in the Peabody/downtown zones.

3.5 Parties: how many riders per booking

Evidence	April's 1.386 van passengers per *completed* booking APRIL EVIDENCE.
Assumption	Party size P(1, 2, 3, 4, 5) = 0.73 / 0.18 / 0.055 / 0.025 / 0.01, mean 1.405 ASSUMED. The two means are different quantities; the second was chosen near the first.
Chosen input	The average-April shape is the base (D-38) accepted. Costs and metrics are passenger-weighted, with request-weighted versions also reported. A 14-seat van can take a party of five only when five seats are free at that point in its route.
Limitation	Only the mean is near the data; the shape is assumed. About 1% of bookings (≈1.7 per 168-request night) are parties of five: that is *potential* XL demand. How many XL trips occur depends on the rule and is SIMULATED.
Sensitivity	A weekend shape (mean 1.49).

3.6 Fleet

Evidence	About 27.6 “All services” vehicles at 6 pm, tapering through the night; the series mixes services APRIL EVIDENCE. Hopkins vans were 14-seat in 2018 (press, as recorded in the decision log).
Assumption	A scaled Homewood system: demand is scaled to the fleet so load per van matches April, rather than simulating 20+ vans (D-30).
Chosen input	Five vans × 14 seats, constant 18:00–21:00, then on duty until all accepted work is resolved (D-36) accepted. A sixth van is tested on the *same* requests (D-37) accepted; scaling demand with the fleet is a separate, later experiment.
Limitation	Fewer requests per square mile than the real system means fewer pooling chances. A larger-fleet check is deferred.
Sensitivity	Fleet level 5 vs 6 vans (a factor of the main experiment); April-shaped tapering with shift ends at the full-night level L7 (the shift-end rule is still open).

3.7 Patience: when a waiting rider gives up

Evidence	“Canceled” ≈ 47% of requests, a mixture of rider cancellations, Lyft hand-offs (inferred at roughly a third, plausibly 23–46%) and rebookings or duplicates (unknown) APRIL EVIDENCE DERIVED.
Assumption	Each request carries one hidden personal deadline = request time + patience, identical across rules and hidden from the dispatcher and the Lyft rule. If no vehicle has reached the curb by then, the rider cancels. The deadline keeps running after a Lyft transfer. A vehicle’s curb arrival ends the clock; in a same-second tie the arrival wins (D-39). Mechanism accepted.
Chosen input	Weibull, shape 1.5 (rising impatience), median 18 minutes: a labelled placeholder ASSUMED provisional. Its scale is to be calibrated so that rider cancellations fall within 50–75% of the dashboard “Canceled” share provisional.
Limitation	Patience is *not identified* by aggregates. The 50–75% range brackets uncertainties but is not exhaustive (it ignores transfers recorded as Completed and duplicate requests). No reaction to displayed ETAs; no rebooking; patience independent of party size and place.
Sensitivity	Shape 1 / 1.5 / 2; the rider share across 50–75%; patience low / mid / high from the identified set; silent abandonment (D-42) as an alternative only.

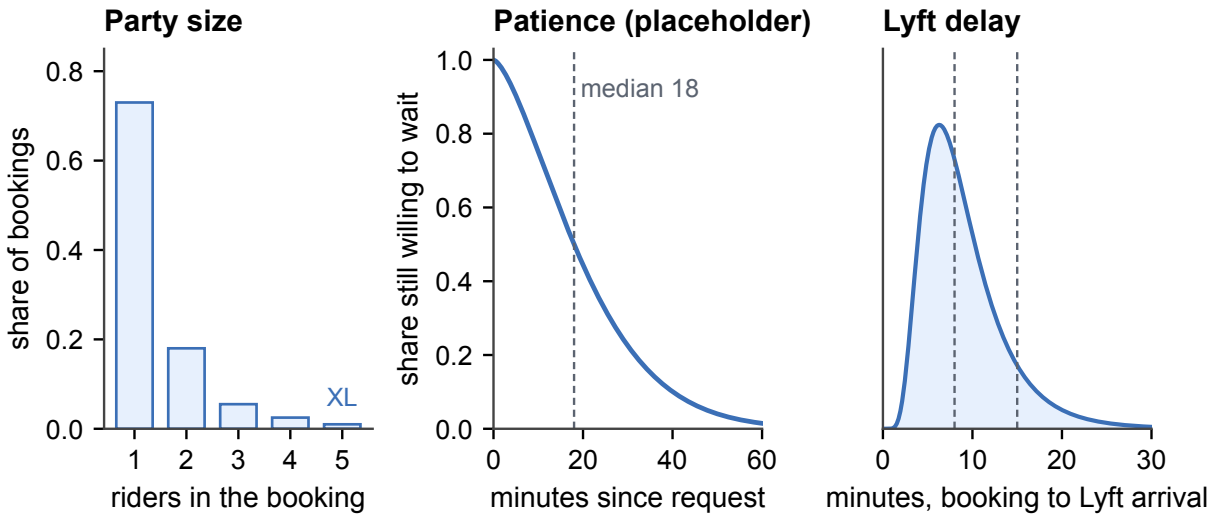


Figure 3: Three assumed input distributions ASSUMED. Left: party-size shares (the base, D-38). Middle: the placeholder patience distribution, shown as the share of riders still willing to wait after a given time. Right: the assumed Lyft delay from booking to arrival; dashed lines mark its median (8 min) and P90 (about 15 min). None of these is a measured Hopkins distribution; the curves are drawn from the stated parameters only.

3.8 No-shows: who is at the curb

Evidence	2,359 "No Show" = 4.7% of *all* recorded requests APRIL EVIDENCE. JHU: the van waits up to 3 minutes; a Lyft waits no more than 3 minutes.
Assumption	Presence is checked only when a vehicle reaches the curb. If the rider is absent, the vehicle waits 180 s and a no-show is recorded; a van no-show frees the van and triggers a re-plan (D-40) accepted. One hidden presence draw U_r per request and *one* probability p for every mode: the rider is absent if $U_r < p$ (D-48(1)). Equality across modes is ASSUMED.
Calculation	Under the status mapping (Completed = van-completed; "No Show" recorded only after a real van arrival; Lyft hand-offs recorded as "Canceled"): van arrivals ≈ 24,095 + 2,359 = 26,454, so $p \approx 2{,}359 / 26{,}454 \approx 8.9\%$ per arrival DERIVED. The arithmetic is exact; the uncertainty is in the mapping.
Chosen input	$p \approx 8.9\%$ proposed, not approved (D-48): it depends on a status mapping the team has not accepted.
Limitation	No-shows are independent of waiting (a labelled simplification); no late boarding. The frozen inputs' "4.7%" labels and `p_noshow: 0.047` fields are all-request shares; the simulator must not read them (errata, D-48(3)).
Sensitivity	Separate van / Lyft / XL thresholds (deferred); presence depending on wait, through silent abandonment (D-42).

Why the denominator matters. April's 4.7% divides by every request, including the roughly 47% that ended "Canceled" before any van came. A probability applied at the curb must divide by the number of curb arrivals instead. Figure 4 shows the two denominators side by side.

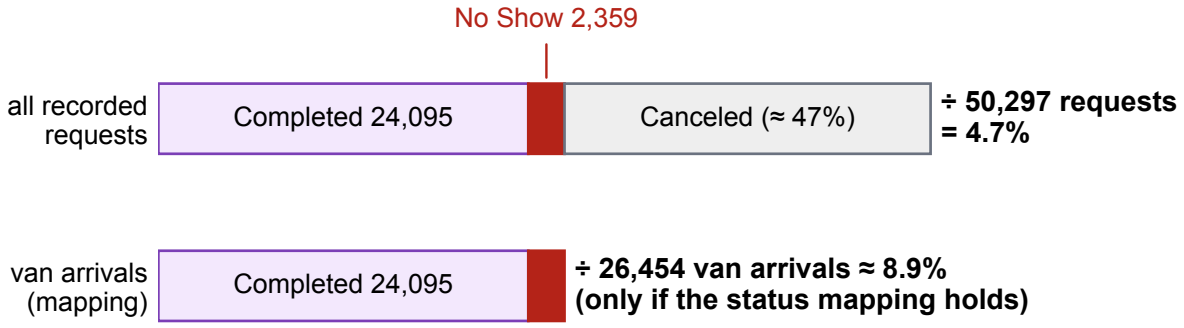


Figure 4: Two denominators for the same 2,359 no-shows. Top: April's share of all recorded requests APRIL EVIDENCE. Bottom: the rate per van arrival, which holds only if every completed ride and every no-show had exactly one van arrival and "Completed" excludes Lyft DERIVED. Applying 4.7% at the curb would use the wrong denominator.

3.9 Lyft: what happens after a transfer

Evidence	JHU page: the dispatcher sends the Lyft, riders are texted, the Lyft waits no more than 3 minutes. Lyft's driver help: a no-show is recorded only after arrival and the wait timer. No Hopkins data on Lyft delays, failures or XL.
Assumption	A lightweight service (D-43) accepted: transfer decision → booking after the dispatcher lag ℓ → arrival after a per-request delay λ_r → the same presence test as for vans → 30 s boarding → a direct, unpooled ride of $t_0(o,d)\,m_L$. No automatic rebooking after a Lyft no-show or failure. XL uses the same settings as standard Lyft by assumption.
Chosen input	$\ell = 2$ minutes ASSUMED; λ_r lognormal, median 8 minutes, P90 ~15 minutes ASSUMED; failure probability $f = 0$ ASSUMED; consent $p_c = 1$, so a decision is a booking request and there is no offer stage (D-45).
Limitation	Everything numerical about Lyft is assumed. Driver cancellations and re-matching are absorbed into the aggregate delay.
Sensitivity	Lyft delay median 5 / 8 / 12 minutes, plus scarcity; lag 0 / 2 / 5 minutes; consent 1.0 / 0.8; dispatcher capacity (at most 3 transfers per 10 minutes); $f > 0$ (`lyft_failed`); longer or scarcer XL.

The rules under study. The Lyft rule is evaluated once per epoch, after the first dispatcher solve, at time τ (Section 4.5). Only riders still pending with the vans (not committed, not on board) are eligible. Every rule counts waiting from the original request, and the rider's patience keeps running after a transfer (spec §6). Table 5 lists the rules; all are kept and run one at a time (D-28) accepted.

Table 5: The Lyft transfer rules (`scenario_spec.md` §6). t_r is the request time; ETAs come from the dispatcher's current plan.

Rule	Transfers a pending rider when	Information used
R0 Never	never (the reference, not the status quo)	none
R1(θ) Elapsed wait	$\tau - t_r \geq \theta$, with $\theta \in \{10, 15\}$ min	request time
R1u(10) Elapsed unassigned	the rider has had no assigned van for ≥ 10 min continuously	assignment status
R2(15) Predicted total wait	predicted pickup $-t_r > 15$ min	plan ETAs
R3(2) Van vs Lyft	predicted van pickup $> \tau + \ell + 8 + 2$ min	ETAs, typical Lyft delay, lag
R4 Congestion-gated (optional)	R2 or R3, only when pending $\div$ available vans $> \kappa$	board totals

R1 always lets the rider wait θ minutes first; R2 can transfer at minute one.

3.10 Patterns, seeds and pairing

Besides the base recipe, three named patterns stress demand only; fleet, policies and the rules for hidden traits are untouched (Table 6). Each pattern has its own demand streams, so comparisons *between* patterns are not paired, while rule comparisons *within* a pattern are (D-49(6)).

Table 6: Demand patterns (`scenario_spec.md` §3.4).

Pattern	What changes	Why it matters for Lyft rules
Base	Nothing	Reference
Burst	40% of the 18:00–19:00 requests inside one 10-minute block	Do rules that act early beat rules that wait?
Far-trip	Trips with an end in Peabody/downtown twice as likely (re-normalised)	Long, isolated trips tie up vans and are natural Lyft candidates
High load	$\rho \times 1.3$	Sustained overload: the resource side of transferring

4 The simulator contract in plain language

The simulator is the referee. Its rules are written so that two people building it independently would produce the *same* event log from the same inputs. The authoritative text is `scenario_spec.md` §17. The conventions come from the 21 October consolidation review (D-46) and are proposed, not approved: written down and used for the build, but not separately approved by the team. A few further conventions were fixed when the simulator was first built (spec §17.9, D-52); they are provisional and summarised in Section 4.10. What has been built and verified is reported in Section 7, not here.

Table 7: Hidden per-request draws (`latent.csv`). All are drawn for every request, used or not, from separate seeded streams ASSUMED.

Draw	Distribution	Used when
`patience_s`	Weibull, shape 1.5, median 18 min (placeholder)	the rider is still waiting for a vehicle when it runs out
`noshow_u`	Uniform U_r; absent if $U_r < p$	any vehicle reaches the curb
`consent_u`	Uniform; accepts Lyft if below p_c (base 1.0)	a transfer is decided
`lyft_delay_s`	Lognormal, median 8 min, P90 ~15 min	a transfer is booked
`lyft_noshow_u`	generated but unused (D-41)	never
`eta_noise`	multiplier on the displayed ETA	ETA-dependent patience variant only

4.1 Time and travel

- The clock counts **integer seconds from 18:00**; every event time is a whole number.
- There is one base travel table $t_0(i, j)$ in matrix seconds. Van time is $t_0 \times m_v$ and Lyft ride time is $t_0 \times m_L$; each multiplier is applied *exactly once*, then rounded to whole seconds with the same rule as the scenario generator (Python's round-half-to-even), so the hand-check answer keys reproduce exactly (D-47(1), a correction of an earlier double application).
- `direct_s` in the request file is the unscaled t_0 and is used only for reporting. The ride guardrail uses $t_0(o, d) \times m_v$.
- Base $m_v = m_L = 1.2$ ASSUMED; the L1–L4 answer keys use $m = 1.0$, as written in their scenario files.

4.2 Request states and commitment

A request moves through named states (Fig. 8 in Section 6):

- `pending_van`: known to the dispatcher, not yet committed; it may be reassigned between vans at every re-plan.
- `committed`: a van has **departed** toward the rider's pickup. Commitment happens at departure, and the van's next stop is then frozen (D-46(4)).
- `onboard`, then `van_completed` at drop-off; or `noshow_van`; or `cancel_pending_van` (possible while `pending_van` or `committed`, that is, until a van reaches the curb).
- On the transfer path: `xfer_hold` (decision made, lag running), then `lyft_booked`, then one of `lyft_completed`, `noshow_lyft`, `cancel_after_transfer` or `lyft_failed`.

An audit invariant must hold at every moment: **no request is ever in a van route and in a Lyft state at the same time**.

4.3 What happens when several events share a second

Integer seconds make ties common. The contract fixes the order by event class (Table 8); ties inside a class are broken by request ID, then van ID.

Table 8: Same-second event order (`scenario_spec.md` §17.3; D-46(2) proposed, not approved).

Order	Class	Notes (spec §17.3)
10	Drop-offs (van and Lyft)	Frees seats before anything else is decided
20	Curb arrivals (van and Lyft)	Ends the patience clock, so an arrival wins a same-second tie with the deadline; the presence test is made here
30	Service ends	Boarding dwell ends; a no-show is recorded after the 180-s curb wait
40	Van on duty / off duty	–
50	New requests	–
60	Patience checks	A rider still `pending_van`, `committed` (van driving to the curb, not yet arrived), `xfer_hold` or `lyft_booked` whose deadline has passed cancels
70	Lyft bookings	`xfer_hold` becomes `lyft_booked` at decision + ℓ
80	Epoch	Van solve → evaluate the Lyft rule once → apply transfers → re-solve once if any transfer
90	Departures	Vans leave toward their next stop; this is the moment of commitment

4.4 Epochs: when the dispatcher re-plans

- There is **one epoch per distinct second** that has a trigger. Triggers are a new request, a cancellation or no-show *of a request in van planning*, a transfer (handled inside the epoch by the re-solve), a van going on or off duty, and a periodic tick every 120 s anchored at $t = 0$ (D-46(3); spec §17.4). **The code differs on one trigger** open: at a van's scheduled off-duty time the simulator only logs the time and opens no epoch (Section 4.7). Lyft-side events (booking, Lyft arrival, Lyft no-show, Lyft drop-off) never trigger an epoch, because they cannot change a van plan (D-52).
- **Zero simulated latency:** a plan computed at time τ takes effect at τ. The solver is limited by deterministic node or iteration budgets; wall-clock time is logged as a diagnostic and never changes a plan, so runs are reproducible on any machine (D-46(5)).
- A request that no route in the current plan serves stays **pending**. It still carries a **tentative ETA**, its cheapest feasible insertion computed outside the solver, and the log records why it is pending (competition with other requests, or the candidate cap). Lyft rules that read ETAs read this value (D-47(3)).

4.5 The Lyft rule pass

How the rule fits into an epoch was made explicit when the rules were run with the dispatcher all evening (D-55; spec §17.9):

- **Once per epoch, no cascade.** The dispatcher solves; the rule is evaluated once on that plan; accepted transfers and declines are applied; the dispatcher re-solves once. The rule is *not* evaluated again on the re-solved plan in the same second.
- **What the rule reads.** For a request assigned in the plan, the plan's ETA. For a pending request, its *tentative ETA*: the cheapest feasible insertion into the chosen routes, computed outside the MILP and ignoring the cap. If no van can feasibly take it, there is no ETA (treated as infinite).
- **Tentative ETA is not the pending price.** The tentative ETA is what the rule reads; the pending price (formulation A) is what the MILP charges for leaving a request waiting. They are computed differently, and **neither is a promised pickup time**.
- **R1u's clock.** The "unassigned since" time of a request is updated from the plan the rule read and again from the re-solved plan.
- **Logged.** When a rule is active, the epoch log keeps the plan the rule read (routes, plan ETAs, and each pending request's price and tentative ETA), what it read for each eligible request, its transfers and declines, and whether a re-solve followed (spec §17.8).

4.6 Transfers and the Lyft service

- **Consent.** In the base, $p_c = 1$, so every decision is a booking request. Where $p_c < 1$ (L4 uses 0.9; the consent sensitivity 0.8), a decision is an *offer*: a rider accepts if and only if `consent_u` $< p_c$, so a draw equal to p_c declines. A declining rider stays with the vans, is never offered again, and the decline is counted.
- **Transfer at the decision.** A consented decision at τ removes the request from van planning *at* τ, atomically, and puts it in `xfer_hold`. Leaving at the booking instead was rejected, because with $\ell = 120$ s equal to the tick a van could be sent to a rider who is about to leave by Lyft.
- **Booking after the lag.** At $\tau + \ell$ the booking is placed. If the rider's deadline falls inside the lag, the outcome is `cancel_after_transfer` and **no booking** is counted.
- **Arrival and ride.** The Lyft arrives at booking + λ_r. The presence test uses the same U_r and p as for vans. A present rider boards (30 s dwell, as for vans) and rides directly; an absent one becomes `noshow_lyft` after 180 s.
- Parties of one to four book a standard Lyft; parties of five book an XL. Every count is kept separately for the two. There is no automatic rebooking.
- **The pickup instant, in both modes,** is the curb arrival of the vehicle the rider boards; the 30-s boarding dwell follows. So the wait is W = curb arrival − request time, and the ride H = drop-off arrival − curb arrival, which includes the dwell. This is the convention of the L1–L4 answer keys (spec §17.5). For riders who do not board, the curb arrival is still recorded, and no wait-to-pickup is reported for them.

4.7 Shifts and closing

After the arrival window, vans stay on duty until every accepted request is resolved (D-36; D-46(6)). For the L5 rehearsal level, the frozen `avail_end_s` = 3600 is read as the end of the arrival window, not a shift end. For tapering cases (L7), the shift-end grace is $g = 0$. Every run must end with `unresolved_close` = 0.

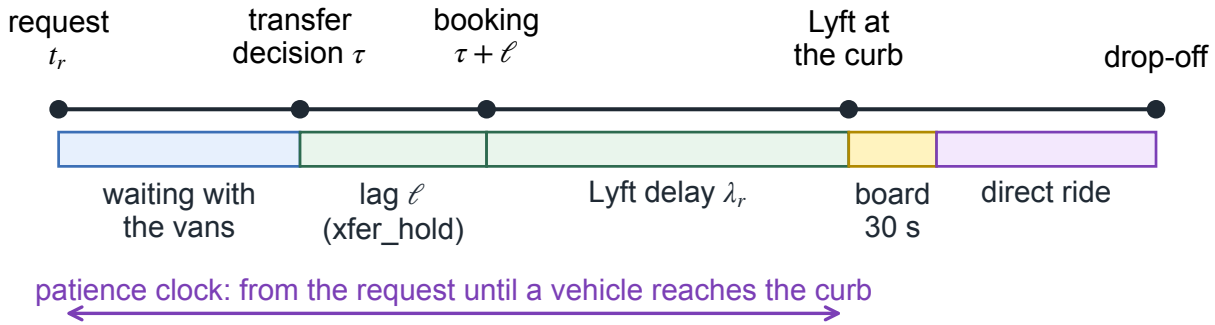


Figure 5: Timeline of a transferred rider (not to scale). The request leaves van planning at the decision; the booking is placed one lag later; the Lyft arrives after the rider's hidden delay λ_r. The rider's patience clock keeps running from the original request until a vehicle reaches the curb. The wait is measured from the original request to that curb arrival, the pickup instant; a transfer never resets the clock.

The shift-end rule itself is open open. The documents give two wordings: a new pickup must *start* before the van's end (spec §17.6), or its *drop-off* must end before it (`model_design.md` §3.3). It matters only for tapering cases (L7), is not implemented, and is to be settled before L7 is built. Riders already on board are always finished.

Off-duty trigger: specification and code differ open. Spec §17.4 and D-46(3) list a van going off duty as an epoch trigger. The simulator's off-duty handler (`src/sim/engine.py`) only logs the off-duty time (as deferred while any request is unresolved) and opens no epoch; it never takes a van off duty, so in effect vans stay on duty until the run ends, consistent with the closing rule. The simulator is unchanged. Whether the difference affected any L1–L5 decision has not been checked; it is to be settled with the shift-end rule before L7 (`scenario_methodology.md` v1.6 §7b).

4.8 Inputs the simulator must not read

- `p_noshow` in `scenario.json` and the "DERIVED rate 4.7%" label (errata, D-48(3)); p comes from the behaviour configuration.
- `lyft_noshow_u` (unused under the shared presence draw, D-41).
- `eta_noise`, except in the ETA-dependent patience variant.

More generally, no policy may see future requests or any hidden draw. The only exception is the clearly labelled hindsight benchmark H on tiny instances (`model_design.md` §3.1).

4.9 Required logs

- **Event log:** every event with time, class, request, van, old state and new state.
- **Epoch log:** the trigger, open requests, candidates per van, solver status and gap, the chosen plan, pending requests with their reason and tentative ETA, and each Lyft decision with the quantity the rule read.

- **Plan audit** after every epoch: capacity, precedence, commitments, shift ends, the guardrail (no new or increased violation) and the transfer invariant.
- **Run manifest:** input hashes, policy file, code version, solver version and settings.

4.10 Conventions fixed when the simulator was built

Building the simulator required a few rules the contract had not spelled out. They are recorded in spec §17.9 and D-52, and are provisional (team review pending):

- **Moving vans** can be re-routed only at a stop; once a van departs toward a stop it completes that leg.
- **Cancelled pickup:** if the rider cancels while the van is driving, the van still reaches the curb, finds no rider and is free at once (no dwell).
- **No-show:** recorded 180 s after curb arrival; the van then leaves without dwell. The simulator never boards an absent rider.
- **Curb-wait planning (D-54, adopted as the planning default, provisional):** while the 180-s timer runs, the pickup stays committed, and *for planning only* the dispatcher assumes the rider may board when the timer expires. It keeps the 30-s dwell, the seat and the drop-off obligation, with the guardrail counted from the expiry. The planner sees this as "not present yet, timer running", distinct from a terminal no-show. The obligation is removed when the no-show actually expires, which triggers an epoch. This is a planning convention, not Hopkins policy and not a late-boarding probability. Once every request is resolved, any remaining plan is void. The earlier M1 view, which treated the no-show as certain during the timer, stays selectable only to reproduce M1.
- **Planning a committed pickup:** the dispatcher plans as if the rider boards, because it cannot see the presence draw.
- **0-s legs** (consecutive stops at the same point): the arrival is processed in the same second, after that second's departures. This is the one exception to the class order of Table 8.
- **Ties inside a class** are broken by key, so request IDs (R…) sort before van IDs (V…).
- **Lyft window:** from M2 on, the rule acts at every epoch of the evening. The M0 harness, in which the L4 rules act only before 18:15 (900 s, the hand-checkable window), stays selectable to reproduce M0 (spec §17.9).
- **L5 settings (D-56):** set in the run configuration, never read from the frozen inputs: lag 2 min, consent 1.0, presence $p = 0.089$ (provisional). The frozen inputs are unchanged.
- **Tie searches (D-52 amendment 5):** if a re-solve that looks for equally good plans fails, the search is labelled incomplete and the run continues; the main solve still stops the run on any solver error. An incomplete tie search does not prove the optimum is unique.
- **Guardrail excesses between epochs (D-57):** plans never create or increase a violation, but a ride may still exceed the guardrail after an unplanned event. The batch check accepts an excess only if it is attributable to van no-shows on the same van during the ride (at most 150 s each, plus a 13-s matrix tolerance); otherwise the run fails.

4.11 Terminal states and the dashboard view

Each request ends in exactly one terminal state SIMULATED: `van_completed`, `lyft_completed`, `cancel_pending_van`, `cancel_after_transfer`, `noshow_van`, `noshow_lyft`, `lyft_failed` (sensitivity only) or `unresolved_close` (must be zero). A transfer is an event, not a completion. Wait is always the pickup instant (curb arrival of the boarding vehicle, either mode) minus request time. For calibration only, a dashboard-comparable view is derived: "Completed" = van-completed; "Canceled" = rider cancellations + transfers (spec §5).

4.12 Designed checks: the level ladder

Before any experiment, the simulator must reproduce hand-computed answers on small, designed cases DESIGNED CHECK. The answer keys were computed by the scenario generator from the travel matrix and recomputed in tests. The M0 simulator now reproduces the L1–L3 keys to the second and the L4 pre-18:15 Lyft key (Section 7). Table 9 summarises the ladder.

The L4 key covers R0 plus five active rule cases: R1(10), R1(15), R1u(10), R2(15) and R3(2). Across those five rule runs there are 22 transfer decisions (R1(10) 3, R1u(10) 3, R2(15) 8, R3(2) 8; 4 of them by XL) and 4 consent declines (one per rule, except R1(15), which makes no decision before 18:15). These are totals across separate rule runs, not counts for one night.

Table 9: The level ladder (`scenario_spec.md` §8). L1–L4 check correctness DESIGNED CHECK; L5 rehearses the pipeline; only L6–L7 answer the research question SIMULATED.

Level	Size	What it checks
L1	2 vans, 16 requests, one open at a time	Every pickup and drop-off time, to the second
L2	L1 + 2 designated no-shows	The 180-s curb wait, no-show handling, re-plan
L3	L1 + designated patience values	The cancel-while-waiting path
L4	2 vans, 16 requests in ~20 min; vans on duty at 18:15; one party of five	Lyft bookkeeping before 18:15: 22 transfers (4 by XL) and 4 declines, summed over five rule runs
L5	2 vans, 28 requests in 60 min, ≥ 40% in a burst; 30 seeds	Dress rehearsal: pairing, metrics, explanations; too small for conclusions
L6	5 vans constant, 168 requests + backlog; sixth van on the same requests	Main Lyft-rule experiments (6 rules × 4 patterns × 2 fleet levels × 30 nights)
L7	5–6 vans, 18:00–02:00 + backlog (counts not yet derived)	Confirmatory runs with taper and shift ends

Stated in advance: L1–L4 should show almost no algorithm differences, because the load is low. Conclusions come only from L6–L7.

5 The dispatcher's mathematics

This section explains how the van dispatcher scores and chooses its plans: first in words, then in symbols, then through a small worked example. It ends with what the word "optimal" can and cannot mean here.

5.1 The idea in words

At each epoch the van dispatcher looks at every van and every rider not yet picked up. It decides who each van picks up next and in what order. Only each van's next move is carried out; the rest is a plan that the next epoch may change. This is *rolling-horizon re-optimization* of a dynamic dial-a-ride problem, a standard framing in the literature (Berbeglia, Cordeau and Laporte 2010; Ho et al. 2018) and consistent with TransLoc's 2016 description of its engine, which re-planned on events and every two minutes.

The dispatcher plans only from what a real dispatcher could see: requests already revealed, van states, travel tables and published rules. It never sees future requests or riders' hidden traits.

5.2 What a "good" plan means

A plan is scored in **points**, which are penalised passenger-minutes; lower is better. The points are *priorities, not money*. Four ideas make up the score.

Waiting counts more the longer someone has waited. Each passenger's total wait W (from the original request, so time already spent waiting counts) is charged a convex penalty $\varphi(W)$: the first five minutes cost 1 point per minute, the next five 2.5, then 5, 10 and finally 20 points per minute beyond 20 minutes (Fig. 6). Thus $\varphi(5) = 5$, $\varphi(10) = 17.5$, $\varphi(15) = 42.5$ and $\varphi(20) = 92.5$. The break-points at 15 and 20 minutes follow the published Homewood targets. Only the 3–10 minute region is constrained by evidence: TransLoc's 2016 example prefers cutting one minute from a 10-minute wait over two minutes from a 5-minute wait, so $\varphi(10) - \varphi(9) > \varphi(5) - \varphi(3)$. A straight line fails that test (1 is not more than 2); the chosen slopes pass it ($2.5 > 2$). The rest of the shape is our choice ASSUMED.

Riding counts, but less than waiting. Each passenger-minute on board costs $\beta = 0.5$ points. Because $\beta = 0.5$ and every slope of φ is at least 1, a waiting minute always outweighs a riding minute, from minute zero (D-47(5), a correction of earlier wording). This expresses "waiting matters more" as a marginal priority, not an absolute one ASSUMED.

Driving costs a little. Each route-kilometre costs $\gamma = 0.1$ points, a small tie-breaker against pointless driving ASSUMED.

Nobody rides unreasonably long. The ride guardrail for request r is

$$G_r = \max\big(25 \text{ min},\ 2\,T_r + 5 \text{ min}\big), \tag{2}$$

where T_r is the direct van time ASSUMED. The form is accepted (D-29). The enforcement convention is proposed, not approved (D-47(2)): no candidate route may *create or increase* a guardrail violation; the "committed work only" route is exempt, and any violation already locked into committed work is charged a very large penalty Ψ per passenger-minute over the guardrail and

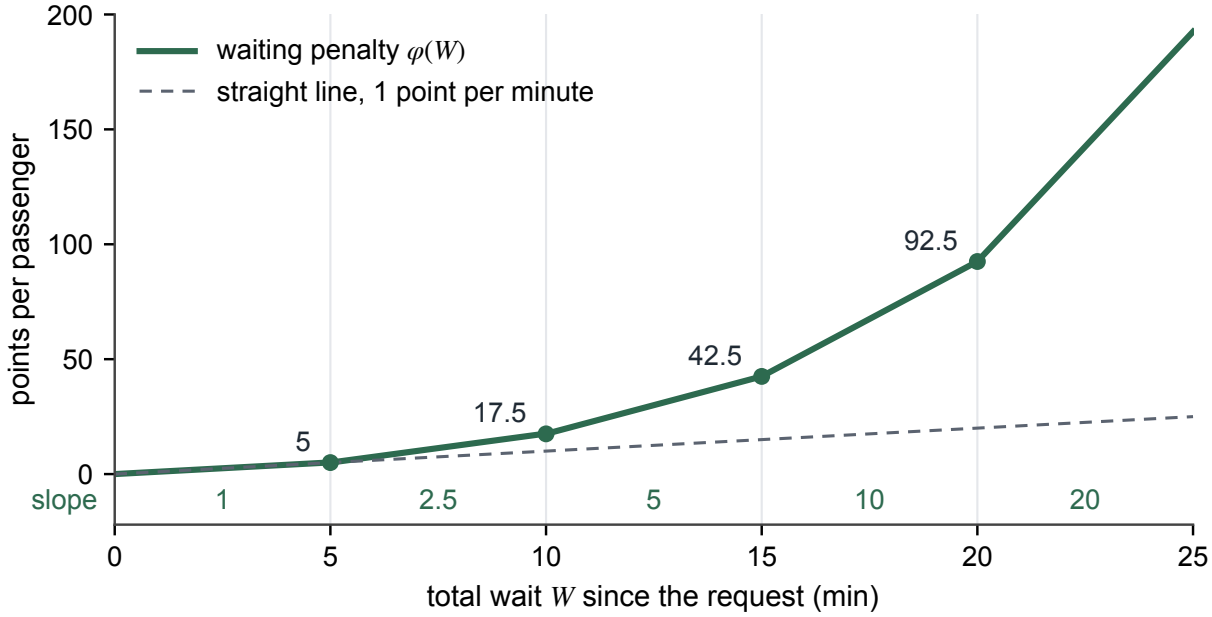


Figure 6: The convex waiting penalty $\varphi(W)$ per passenger (D-29 accepted). The slope rises at 5, 10, 15 and 20 minutes, so a minute of waiting costs more the longer the rider has already waited. The dashed line shows a constant 1 point per minute for comparison.

logged. The 20-minute published target is a reporting threshold, not the guardrail, because a hard 20 would be stricter than the service delivers (April's ride P90 is about 24 minutes).

Leaving a request pending has a price. A request left unassigned for this epoch costs

$$\pi_r = q_r \, \varphi\big(W_r^{\text{next}}\big) + M, \tag{3}$$

where W_r^{next} is its wait if picked up no earlier than the next epoch and M is large. With a large M the solver first serves as many requests as it can within its candidate routes, then minimises points. A request can still be left pending when routes compete for it (taking it would mean dropping two others) or when the candidate cap leaves it out (D-47(3), a correction of an earlier claim that this could not happen). M must be the same for every dispatcher, otherwise a dispatcher could look good by starving hard requests. In the implementation M is set at each epoch to an explicit bound that guarantees every optimal plan leaves the fewest requests pending that the candidate set allows (D-52).

A known bias, and three ways to price a pending request (D-53). The price above charges a pending request as if it were picked up at the next epoch, earlier than any physically possible pickup, while a served request is charged its real, later pickup. When the candidate cap binds, the cheapest requests to defer can therefore be the longest waiters, which works against the purpose of the convex φ. The implementation offers three selectable formulations; A is now the default. In each, $\pi_r = b_r + M$:

- **Original:** $b_r = q_r \, \varphi(t_{\text{next}} - t_r)$, the wait at the next epoch. This was the documented price (D-29) and the M1 default; it remains an option to reproduce M1.
- **A, earliest physical pickup:** $b_r = q_r \, \varphi(\max(t_{\text{next}}, L_r) - t_r)$, where L_r is the earliest time any on-duty van could physically reach the rider directly from where it will next be free. A is a

lower bound on any future service, with small stated exceptions: a triangle-inequality slack of at most 13 s in the travel table, vans not yet on duty, and the next-epoch floor. It is still optimistic when the nearest van is busy with its own chosen route. It adds no variables.

- **B, after a chosen route:** b_r is the wait the rider would have if picked up at the end of one of the chosen routes, the cheapest such route being used. B is an *estimate, not a bound*. It can under-charge, when several pending riders claim the same van's free time, and over-charge, when a later epoch inserts the rider earlier. Its size grows with the number of requests times the number of candidate routes.

The same M bound is valid for all three. It guarantees the fewest pending requests only for solutions proven optimal. **Neither A nor B describes a feasible joint future schedule** for all pending riders. The experiment behind the choice is in Section 7.

The decision: A, approved provisionally and now the default provisional. There were two separate approvals: first the team authorized only an *experiment* comparing the three formulations, and then it approved A (D-53 approval; design trail §11). The reasons are:

- A replaces next-tick-only pricing with a direct physical pickup estimate, L_r = the minimum over on-duty vans of (free time + direct travel to the origin);
- it adds no variables to the MILP;
- the verified M bound keeps the intended request-count priority for proven-optimal solutions;
- B grew the model substantially (about 9,800 extra variables on L4, about 135,000 in a synthetic five-van epoch) without giving a reliable estimate of deferred service, since it can err in both directions.

The approval does *not* rest on any single rider's earlier pickup or on the lower maximum wait of one evening.

Runtime. A added no variables, and its MILP solve time was essentially unchanged from the original (0.10 s versus 0.10 s per evening). The whole evening ran longer (about 35 s versus 25 s) only because candidate generation followed a different trajectory, not because of A.

Limitations that remain. A ignores the competing work of the van that could arrive first, so it can still under-price postponement. It does not guarantee oldest-first service, accurate future pickup times, or a feasible joint schedule for all pending requests. "Earliest physical pickup" rests on the implementation's assumptions: only vans already on duty count, travel is direct with no detours, the price is floored at the next tick, and there is a triangle-inequality exception of at most 13 s on 3 matrix pairs. On L4 the change *redistributed* waiting: the longest waits fell and some younger requests waited longer. It did not improve every outcome or establish general superiority. The choice is to be revisited if the five-van benchmark (M5) shows the remaining bias matters, if the team changes the request-count priority, or if a cheaper and reliable estimate of deferred service appears.

5.3 The route-selection model

Candidate routes. For each van v, routes k are built in advance. Each is an ordered stop list that starts at the van's committed next stop, keeps all drop-offs of riders already on board

and any committed pickup, and adds a subset S_k of open requests (pickup before drop-off). The "committed work only" route is always included, so a feasible plan always exists. Capacity, pickup-before-drop-off, commitments, shift ends and the guardrail are all enforced while routes are built; each candidate is simulated forward to give every rider's pickup and drop-off times.

Route cost. With q_r passengers, wait W_r and ride time H_r:

$$c_{vk} = \sum_{r \in S_k \cup C_v} q_r \, \varphi(W_r) \;+\; \beta \sum_{r \text{ on route}} q_r H_r \;+\; \gamma \, \mathrm{km}_k \;+\; \Psi \sum_r q_r \big(H_r - G_r\big)^+ . \tag{4}$$

C_v is the van's committed pickup. Because φ is evaluated while route costs are computed, outside the solver, the optimization stays linear whatever shape φ has.

The choice. With $y_{vk} = 1$ if van v uses route k, and $u_r = 1$ if open request r stays pending:

$$\begin{aligned} \min \; & \sum_v \sum_k c_{vk}\, y_{vk} + \sum_r \pi_r \, u_r \\ \text{s.t.} \; & \textstyle\sum_k y_{vk} = 1 && \text{for every van } v \quad \text{(one route per van)} \\ & \textstyle\sum_{v,k:\; r \in S_k} y_{vk} + u_r = 1 && \text{for every open request } r \quad \text{(one van, or pending)} \\ & y_{vk},\; u_r \in \{0,1\}. \end{aligned} \tag{5}$$

This is the pattern of Alonso-Mora et al. (2017), with "drop the request" replaced by "defer it at price π_r".

The candidate set. For tractability, each candidate route may add only a few open requests, and every open request gets at least one single-insertion candidate on every van that can feasibly take it (D-47(3)). The settings used for milestones M0–M1 are recorded in D-52: every subset of at most $K = 3$ open requests (new pickups) per van, each subset getting its best stop order found exactly. Under the D-52 amendment that best order covers all of the van's stops, including the drop-offs of riders already on board. D-52 also replaces the earlier wording that counted the cap relative to the van's previous plan. These settings are implemented and provisional (team review pending); they change the candidate set, not the objective.

5.4 Worked example: why the long waiter goes first

One empty van, two open requests, both routes 10 km, dwell omitted for clarity. Request r1 has one passenger who has already waited 8 minutes; r2 has two passengers who have waited 1 minute. The example is taken from methodology §6.

The dispatcher picks A. The rider who has already waited 8 minutes goes first, even though r2 carries two passengers and B gives slightly less riding. Passenger weighting gives a group's combined delay more weight, but it does not automatically put groups first: the plan with the lowest total wins. The convex φ is what protects the long waiter: in B, r1's wait crosses 15 minutes, where each minute costs 10 points. Both rides are far below the 25-minute guardrail. This is a **DESIGNED CHECK** illustration computed by hand, not a simulation output.

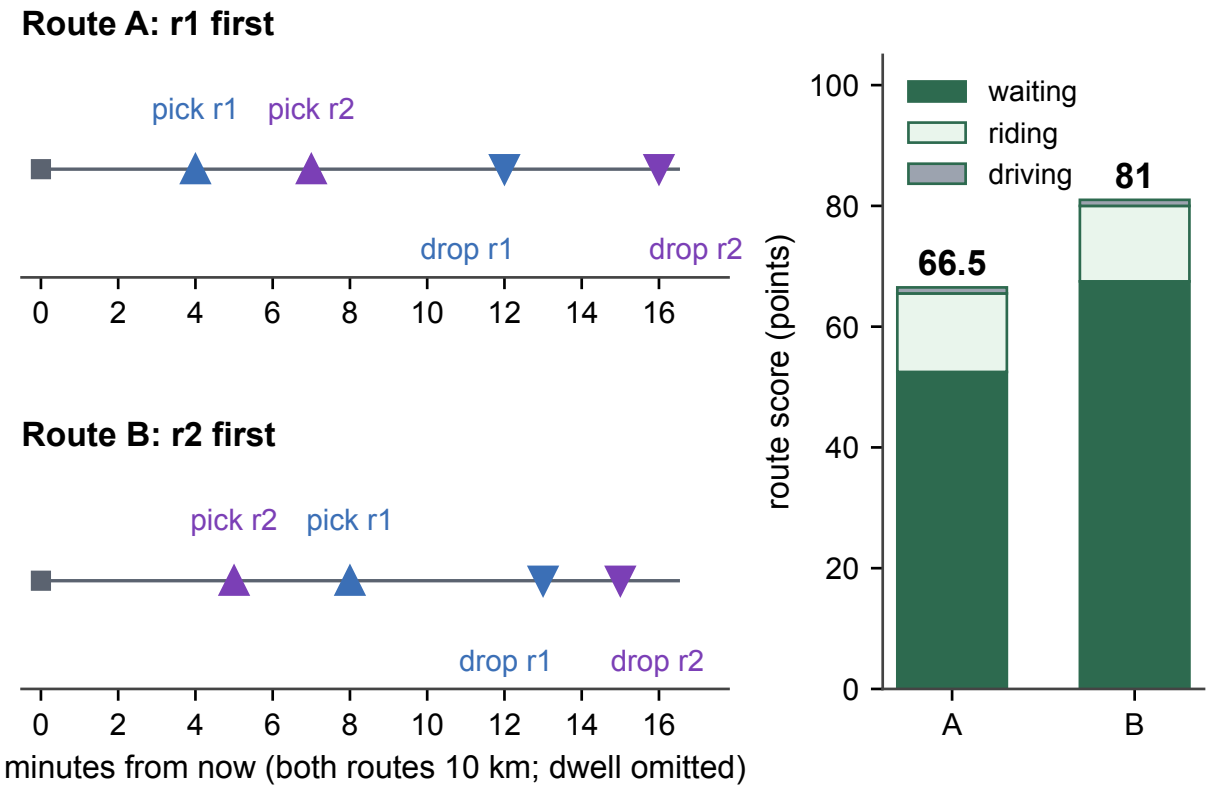


Figure 7: Worked example: two candidate routes for one van. Triangles pointing up are pickups, down are drop-offs. Right: each route's score split into waiting, riding and driving points. The arithmetic is in Table 10.

Table 10: Worked example arithmetic. Times in minutes; $\varphi(12) = 5{+}12.5{+}10 = 27.5$, $\varphi(8) = 5{+}7.5 = 12.5$, $\varphi(16) = 42.5 + 10 = 52.5$, $\varphi(6) = 5 + 2.5 = 7.5$. All values checked for this document.

	Route A: r1 first	**Route B: r2 first**
Stops	r1 at 4, r2 at 7, drop r1 at 12, drop r2 at 16	r2 at 5, r1 at 8, drop r1 at 13, drop r2 at 15
Waits	$W_1 = 8 + 4 = 12$; $W_2 = 1 + 7 = 8$	$W_1 = 8 + 8 = 16$; $W_2 = 1 + 5 = 6$
Rides	$H_1 = 12 - 4 = 8$; $H_2 = 16 - 7 = 9$	$H_1 = 13 - 8 = 5$; $H_2 = 15 - 5 = 10$
Waiting	$1 \cdot 27.5 + 2 \cdot 12.5 = 52.5$	$1 \cdot 52.5 + 2 \cdot 7.5 = 67.5$
Riding	$0.5\,(1 \cdot 8 + 2 \cdot 9) = 13.0$	$0.5\,(1 \cdot 5 + 2 \cdot 10) = 12.5$
Driving	$0.1 \cdot 10 = 1.0$	$0.1 \cdot 10 = 1.0$
Total	**66.5**	**81.0**

5.5 What “optimal” means here

The restricted route-selection model C is **optimal over its candidate set, subject to solver status**. It is not “optimal over all routes”, and not “optimal for the evening”.

Three reasons keep the claim narrow:

- **Candidate set.** C chooses only among the routes that were built. A route outside the set is never considered.
- **Solver status.** Optimality holds only when the solver closes its gap. The proposed settings are a relative gap of 0, a small absolute gap and a node limit, with M and Ψ set and recorded before any comparison (D-47(4) proposed, not approved). The values used in M0–M1 (D-52 and amendments 2–3, provisional) are: HiGHS with one thread and a fixed seed, relative gap 0, absolute gap below one cost unit, a node limit of 100,000 as the only budget, and $\Psi = 1000$ points per passenger-minute. A wall-clock safety limit stops the run with an error rather than letting wall time change a plan. If the node budget is reached, the best plan found is executed and labelled *feasible, unproven*, and no pending reasons are established for that epoch. A solve that ends with no plan at all (*failed*) executes committed work only and is logged. A solver error or an infeasible model stops the run (infeasibility cannot occur, because the committed-only routes are always feasible); T is never invoked. None of these cases occurred in the M0–M1 runs. An epoch with nothing to decide is labelled *empty*. Status and gap are logged at every epoch, and a plan audit after every epoch stops the run on any violation (D-52 amendment 3).
- **One epoch at a time.** Each epoch is solved with what is known then. Per-epoch optimality is not whole-evening optimality.

A difference between C and the heuristic T measures the search alone only when both search the same candidate set with the same objective (D-47(5)).

5.6 The dispatchers and the held-fixed choice

Status (D-51). C is provisional: the team *intends* to hold it fixed in the main Lyft-rule experiments, conditional on a later benchmark showing it fits the time budget on the full workload. It does not claim to reproduce Hopkins’ proprietary dispatcher; it shares the published structure (event plus periodic re-planning, reassignment, convex waiting penalty). **T is a development contingency, never an automatic runtime fallback.** If C proves unsuitable, the team decides whether to change the fixed dispatcher before experiments are frozen.

The Lyft rule is a separate module. It runs after each dispatcher solve and reads the board: each open request’s request time, party size, assignment state and predicted van pickup. Keeping it separate mirrors Hopkins’ human dispatcher, keeps policy apart from algorithm, stays interpretable and needs no price for a Lyft trip. An integrated variant (Lyft as a third option inside the solve, as in Grahn et al. 2023) is deferred.

Table 11: Van dispatchers (`model_design.md` §6.1, with the D-51 update).

Dispatcher	Definition	Role
C, route selection	The model above, solved with HiGHS each epoch over a recorded candidate set	Provisionally the held-fixed dispatcher (D-51); built and verified on L1–L4
T, TransLoc-like	Same model, solved by insertion and local search with a fixed iteration budget	Later robustness variant or development contingency; not built
T−	T without reassignment and/or a 300-s cadence	Baseline-uncertainty variant
G, greedy	Insert each new request at its cheapest position on arrival; never reassign	Floor; robustness check
H, hindsight	Node-arc model with full knowledge of a tiny night	Information bound on L1 only

Weighted rather than lexicographic. A strict "waiting first, always" ordering risks unbounded on-board detours; the weighted form trades them at an explicit, swept rate ($\beta = 0.25 / 0.5 / 1$). The lexicographic form is kept as a variant to show the frontier (`model_design.md` §4.2).

6 One request's lifecycle

This section follows a single request through the system. It describes the rules, not a simulated night: no times or outcomes below are SIMULATED.

6.1 The van path

1. **Request.** The request is revealed at its request time and joins the next epoch as `pending_van`. Its hidden traits (patience, presence, consent, Lyft delay) are already fixed in the scenario file but stay invisible to the dispatcher and the Lyft rule.
2. **Planning.** At every epoch the dispatcher may place it on a van's route, move it to another van, or leave it pending with a tentative ETA. The Lyft rule then reads its waiting time and ETA.
3. **Commitment.** When a van departs toward its pickup, the request becomes `committed`: the van's next stop is frozen and it can no longer be reassigned or transferred.
4. **Curb arrival and presence test.** When the van reaches the curb, the patience clock stops; this curb arrival is the pickup instant that ends the wait. The simulator compares the rider's presence draw U_r with p. Present: the rider boards (30 s dwell) and becomes `onboard`. Absent: the van waits 180 s, the outcome is `noshow_van`, the van leaves without dwell and the dispatcher re-plans. While the timer runs, the pickup stays committed and the dispatcher keeps a possible boarding at expiry in its plan; that planned obligation is removed when the no-show expires (D-54).

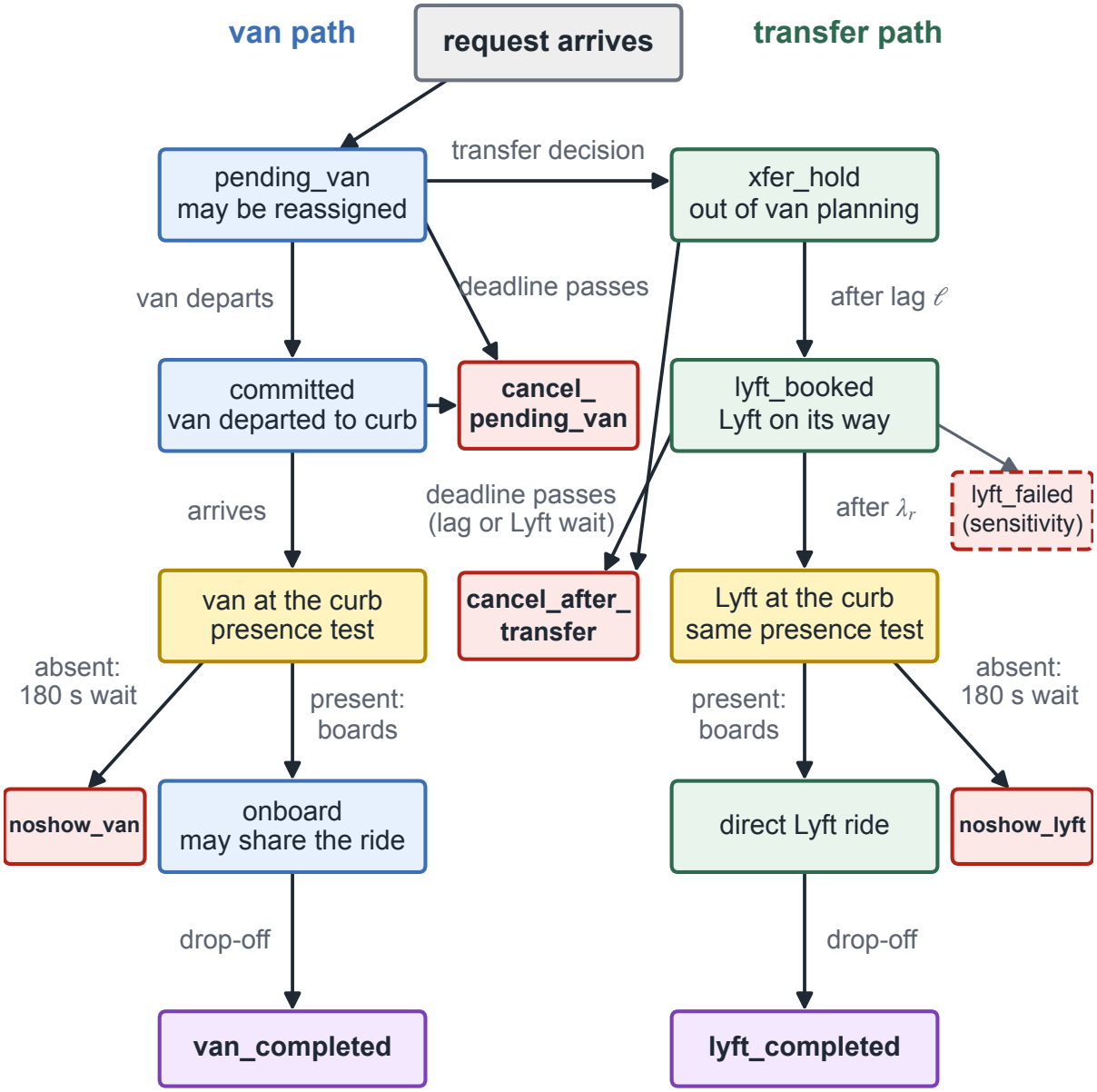


Figure 8: The life of one request. Left: the van path. Right: the transfer path. Red boxes are terminal outcomes; the dashed box is used only in a sensitivity case. "Deadline passes" is the hidden patience deadline. Arrows follow the states and patience checks of `scenario_spec.md` §17.2–17.3; a rider can cancel both while still `pending_van` and while `committed` (van driving to the curb). During the 180-s wait for an absent rider, the dispatcher plans as if the rider might still board at expiry (D-54); this is a planning convention only, and the simulator never boards an absent rider.

5. **Cancellation.** If the rider's deadline passes while still `pending_van`, or while `committed` with the van on its way, the outcome is `cancel_pending_van`. A van already driving still reaches the empty curb and is then free at once (spec §17.9).
6. **Drop-off.** The van may pick up others on the way, within capacity and the guardrail. At drop-off the outcome is `van_completed`; the wait is curb arrival minus request time, and the ride is drop-off minus curb arrival (it includes the boarding dwell).

6.2 The transfer path

1. **Decision.** After a dispatcher solve, the Lyft rule may decide to transfer the pending request. In the base every decision is accepted; with $p_c < 1$ the rider may decline and stay with the vans.
2. **Hold.** The request leaves van planning at once (`xfer_hold`). If the deadline passes during the lag: `cancel_after_transfer`, and no booking is counted.
3. **Booking.** One lag later the Lyft is booked (`lyft_booked`), standard or XL by party size.
4. **Arrival.** The Lyft reaches the curb after the hidden delay λ_r; the patience clock stops; the same presence test applies. Absent: `noshow_lyft` after 180 s. If the deadline passes before arrival: `cancel_after_transfer`.
5. **Ride.** The rider boards and rides directly; the outcome is `lyft_completed`. The wait still counts from the original request.

6.3 Where each hidden trait enters

Table 12: Where each hidden trait turns into an outcome.

Hidden trait	Applied at	Possible outcome
Patience deadline	Patience checks (order 60) while waiting	`cancel_pending_van` or `cancel_after_transfer`
Presence draw U_r	The first curb arrival of any vehicle	boarding, or a no-show after 180 s
Consent draw	A transfer decision, only when $p_c < 1$	stays with vans; decline counted
Lyft delay λ_r	Booking	Lyft curb arrival time

Because these traits are drawn once per request and shared by every rule, a rider who would have been absent at the curb is absent whichever vehicle comes. Only the waits that a rule produces can change whether that rider is still there to be tested.

7 Implementation status, provisional assumptions and next milestones

This section reports what the simulator milestones M0–M2 and the M3 batch (22 October 2026) built and verified, taken from `scenario_methodology.md` v1.6 §7–§9. It keeps two things apart: what has been **verified** and what is only **planned**.

What this section does and does not show

Only designed cases have been simulated: the validation levels L1–L4 and the L5 rehearsal nights. The outcomes below are SIMULATED outcomes of DESIGNED CHECK cases, not evidence about the real service. M2 ran every Lyft rule through one designed integration evening, and M3 replayed 30 two-van rehearsal nights under six rules, to check that the pieces work together and to describe variability. **M3 is a descriptive comparison of the six rules on two-van rehearsal nights; no intervals have been computed and no rule is ranked** (D-50 open).

7.1 What was built

The code lives in `src/sim/`, with settings recorded in D-52 and its amendments:

- **Input loader:** reads the frozen scenario files read-only and refuses to run if a content hash differs from the frozen record.
- **Event engine (the referee):** owns the clock, the same-second order of Table 8, every request state and the hidden rider traits, which it applies only when their event happens.
- **Dispatcher C:** for each van, every subset of at most three open requests with its best order of all stops (found exactly), then the selection model of Section 5 solved by HiGHS, with the explicit M bound, exact integer costs, status handling and a fixed tie rule.
- **Lyft rules:** from M2 they act at every epoch of the L4 evening, with the full transfer lifecycle (D-55); the M0 window (before 18:15 only) stays selectable to reproduce M0.
- **Runner:** writes the event log, the epoch log, per-request outcomes, a run manifest and a readable narrative.

7.2 What was verified

Table 13: What M0–M1 verified (`scenario_methodology.md` v1.6 §7).

Check	Result
L1–L3 answer keys	Every request's van, curb arrival, pickup, wait, drop-off, ride, no-show time and cancellation time reproduced **to the second**, compared with the hand-check sheets by two independent test files
L4 Lyft mechanics before 18:15	For each of the five active rule cases, the transfer and decline sets, decision, booking (+120 s), Lyft pickup (+480 s), wait, drop-off and XL class match: 22 transfers (4 XL) and 4 declines in total across the five rule runs
Route selection	All 46 L4 R0 epochs and all L1–L3 epochs solved to proven optimality over the candidate set. Under the current candidate default, the MILP's choice equals full enumeration of compatible candidate combinations on all 46 L4 R0 epochs (151,710 combinations), including the tied epoch and the fewest pending requests. An earlier independent math review found the same on its own runs, made under the earlier fixed drop order
Best order per subset	For 40 van states of the L4 R0 run, every subset of at most two requests and a sample of 80 subsets of three (1,130 subsets, 113,879 stop orders): each candidate's cost equals the brute-force minimum over all feasible orders. Not every subset of three was brute-forced
Invariants	One terminal state per request; no request both in a van route and a Lyft state; capacity; committed pickups served by the committing van; no ride over the guardrail in these runs
Hidden traits	The dispatcher's observation has no hidden-trait fields; changing patience values that never fire leaves plans unchanged
Reproducibility	Two runs give identical event logs and epoch records (wall times excluded); frozen inputs unchanged
Test suite	253 tests pass on 22 October, after M2: 46 pre-existing scenario tests; 39 engine and curb-wait tests; 38 independent route tests, 29 independent pending-price tests and 101 independent M2 integration tests
Integrated review	Ready to hand to the team once its should-fix items were addressed; all were addressed on 22 October (status labels, plan audit, wording)

7.3 What M1 showed on L4 under R0 (original pending price)

L4 is a designed case: two vans come on duty at 18:15, when nine requests are already waiting. The M1 run used the pending price then in force, the original one. Under R0 (no transfers), it showed the following SIMULATED:

- At 18:15 the MILP jointly assigns three requests to each van (the cap $K = 3$), sequences them, and leaves three pending, with the cap recorded as the reason (Table 14).
- The plan changes at most epochs while the queue lasts, because uncommitted requests are reassigned; committed pickups never move.
- All 16 riders are delivered by 19:10. Waits range from 1.2 to 44.4 minutes; the longest ride is 24.9 minutes, within the 25-minute guardrail.

- Run time was about 27 s for the evening (slowest epoch about 8 s). On one L5 night (28 requests with a burst) all 86 epochs were solved to optimality. Wall times are diagnostics only; simulated solver latency is zero.

Table 14: The plan chosen at 18:15 in L4 under R0 DESIGNED CHECK SIMULATED. Times are the curb arrivals planned at 18:15; later epochs revised some of them. Points use φ and the party size. The three pending requests are priced at the next tick (18:16:00), excluding M. Re-derived for this document from the preserved M1 run, `outputs/sim/L4_R0_M1_reference/epochs.jsonl`; they match that run's narrative. A mirror plan (the vans swapped) scores the same.

Van	Request	Party	Requested	Planned curb	Wait	Points
V1	R001	3	18:00:00	18:17:44	17:44	209.50
V1	R003	1	18:00:20	18:21:33	21:13	116.83
V1	R008	1	18:09:40	18:25:17	15:37	48.67
V2	R004	2	18:01:40	18:18:48	17:08	127.67
V2	R007	1	18:09:40	18:21:48	12:08	28.17
V2	R009	1	18:13:50	18:23:08	9:18	15.75
pending	R002	5	18:00:00	(next tick)	16:00	262.50
pending	R005	1	18:06:40	(next tick)	9:20	15.83
pending	R006	2	18:09:30	(next tick)	6:30	17.50
Waiting 546.58 + riding 53.25 + driving 1.65 + pending price 295.83						**897.32**

7.4 A property of the objective that M1 exposed

The original pending price of Section 5 (D-53) shows up directly in Table 14. R002, a party of five that has already waited 15 minutes, is left pending, while R009, waiting about a minute, is served. The reason is arithmetic, not a bug. Pending, R002 is charged as if picked up at the next tick: $5 \times \varphi(16) = 262.5$ points. Its earliest physically possible pickup, 18:19:28, would mean a wait of 19.47 minutes and $5 \times \varphi(19.47) \approx 435.8$ points, about 173 points more. Under the cap, deferring R002 looks cheap. In the run, R002 eventually waited 22:18 and R005 44:22.

7.5 The pending-price decision experiment

On 22 October the team authorized a bounded experiment, not an adoption (D-53 update 2). The three formulations of Section 5 were each run through the whole L4 evening under R0, every arm evolving from its own decisions. Everything else was identical: inputs, fleet, $K = 3$, φ, β, γ, guardrail, solver settings and the curb-wait convention. An independent review checked the implementation and proved the M bound for all three. Every arm delivered all 16 riders with no cancellations, no no-shows and no guardrail violations, and every epoch was proven optimal over its candidate set. The original arm reproduces the M1 trace exactly.

Table 15: Pending-price formulations compared on one designed night (L4 under R0, 16 requests) DESIGNED CHECK SIMULATED. These numbers describe behaviour on this night; they are not a ranking, and no Lyft rule was involved. Waits are in minutes over the 16 riders picked up. Re-derived for this document from the three runs' outcome and epoch files in `outputs/sim/d53_experiment/`; assignment changes are quoted from its `d53_comparison.md`.

Measure	Original	A	B
Wait median / P90 / max	17.1 / 31.0 / 44.4	18.6 / 31.2 / 35.6	21.1 / 34.3 / 38.6
Mean wait	18.8	18.8	21.8
Wait of R002 / R005 / R009	22:18 / 44:22 / 9:48	19:28 / 31:58 / 35:34	20:39 / 32:58 / 35:33
Assignment changes	29	27	35
MILP time (s) / extra variables	0.10 / 0	0.10 / 0	4.8 / up to 9,828

How to read it. Removing the impossible part of the bias shortens the longest waits: R005 waited 44 minutes under the original price and about 32–33 under A or B. The cost is that waiting shifts toward younger requests. R009, the newest request at 18:15, waited 9:48 under the original price and about 35 minutes under A or B. That is the priority the convex penalty intends, but it is a trade-off and is stated as one. B is larger and slower: in a synthetic five-van, 20-request epoch it needed 135,100 extra variables and 19.6 s.

Outcome: A approved provisionally and made the default provisional. The team then approved A as the pending-price formulation (D-53 approval), for the reasons given in Section 5: its meaning and its cost, not this one night. The original price and B remain explicit options, to reproduce M1 and the experiment. The M1 outputs and the experiment's outputs are preserved unchanged.

7.6 M2: the Lyft rules with the dispatcher, over the whole L4 evening

In M2 (D-55) each rule ran separately on the same frozen L4 inputs, with R0 as the reference. Dispatcher C used pending price A and the curb-wait convention, and the L4 settings were a lag of 2 minutes, a Lyft delay of 8 minutes and a consent probability of 0.9. All the following held:

- every epoch was proven optimal over its candidate set (one epoch each under R2 and R3 had nothing to decide);
- every request ended in exactly one terminal state;
- offers, accepted transfers, bookings and completed Lyft trips reconcile, standard and XL separately, and the party of five is always one XL booking;
- the R0 run reproduces the experiment's A arm exactly.

Table 16: M2 on L4: each rule run separately on one designed integration evening (16 requests, 29 passengers) DESIGNED CHECK SIMULATED. **These numbers show that the pieces work together and that every rider and booking is counted once; they do not rank the rules.** Waits are in minutes over the 16 requests picked up, in either mode. "Reached" counts every request whose vehicle reached the curb within 15 minutes. Re-derived for this document from each run's outcome and event files; they match `outputs/sim/m2_L4_report.md`.

Measure	R0	R1(10)	R1(15)	R1u(10)	R2(15)	R3(2)
Van / Lyft completed	16 / 0	9 / 7	11 / 5	13 / 3	4 / 12	4 / 12
Offers accepted / declined	0 / 0	7 / 1	5 / 1	3 / 1	12 / 1	12 / 1
Transfers before / from 18:15	0 / 0	3 / 4	0 / 5	3 / 0	8 / 4	8 / 4
Wait P90	31.2	20.3	25.4	22.9	13.2	11.8
Reached within 15 min (of 16)	5	7	6	7	15	15
Van-km	38.0	24.6	24.9	33.8	16.5	16.5

What the logs show (thresholds unchanged):

- **R1u(10) transfers nobody after 18:15.** Once the vans are on duty, C puts every waiting request into some van's plan within at most 4 minutes, so "continuously unassigned for 10 minutes" never holds. The rule measures assignment, not waiting; its three transfers all happen before 18:15, when no van exists.
- **Elapsed-time rules can send a rider to Lyft whose van was due sooner.** Under R1(15), R001 is transferred at 18:15 although its van was predicted at 18:17:44 and the Lyft arrives at 18:25. That is what a rule based only on time waited does; it is not a defect.
- **R2(15) and R3(2) are similar, not identical.** They transfer R011 at 18:20 and at 18:17 respectively. Their summary rows coincide partly by chance on this small evening.

One transfer, end to end (R2(15) at 18:20). The dispatcher solves first. The rule then reads R011's plan ETA, a pickup at 18:35:20, which would mean a total wait of 18:50 from its 18:16:30 request, more than 15 minutes, so the rule fires. It transfers R011, R012, R015 and R016, each of which leaves van planning at once, and the dispatcher re-solves once. Table 17 shows what the re-solve changed for the riders who stayed on the vans. These are *planned* times at that epoch, read from the epoch log, not promises.

Table 17: Planned drop-offs of riders who stayed on van V1, before and after the 18:20 re-solve under R2(15) DESIGNED CHECK SIMULATED. Planned times at that epoch, not promises. Checked against the plan the rule read and the re-solved plan in `outputs/sim/L4_R2-15/epochs.jsonl`.

Rider	Plan the rule read	Re-solved plan
R013	18:45:59	18:32:30
R003	18:42:57	18:35:18
R014	18:38:18	18:39:31

Limitations of M2. This is one designed evening of 16 requests with no cancellations or no-shows in its frozen inputs; those paths are covered only by controlled in-memory test cases. The

no-cascade rule is confirmed structurally, but L4 never produced a case where a second rule pass would have fired. Nothing here ranks the rules.

7.7 M3: the L5 pilot and the 30-night batch

What these numbers are

Everything in this subsection is a SIMULATED outcome on DESIGNED CHECK rehearsal nights, under provisional assumptions. The nights are **30 synthetic nights replayed under six rules**, not 180 independent nights. They are **two-van** rehearsal nights, not the proposed five-van evening (L6), which is designed but not run. They describe behaviour; they do not rank the rules.

Design (D-56, D-57). L5 is a designed rehearsal level: 30 synthetic one-hour nights, each with 2 vans and 28 requests, 40% of them in a 10-minute burst, with random patience, presence and Lyft delays. Each night is replayed under all six rules with identical riders and hidden traits. The settings, approved for the pilot and provisional, are:

- dispatcher C with $K = 3$, pending price A and the curb-wait convention;
- a dispatcher lag of 2 minutes and consent 1.0 (automatic acceptance, a simplification);
- presence $p = 0.089$ and the frozen placeholder patience.

These settings live in the run configuration, because L5's frozen inputs specify no lag or consent; the deprecated frozen `p_noshow` is not read (spec §17.9). A one-night pilot under R0 and R2(15) came first (D-56). It found and fixed one defect: a solver error during the search for equally good plans now labels that search incomplete instead of stopping the run (D-52 amendment 5).

Verification. All 180 runs passed every check, and inputs and configuration were identical across the six arms on all 30 nights.

- **Solves.** All 13,663 main solves were proven optimal over their candidate sets.
- **Tie searches.** 41 epochs had an incomplete search for equally good plans. For all 41, full enumeration shows a unique optimum, which is the plan chosen.
- **Reproducibility.** Six sampled reruns were byte-identical to the batch outputs, and the two r00 batch runs were byte-identical to the earlier pilot outputs.

Table 18: What happened to transferred riders: totals over the 30 L5 nights DESIGNED CHECK SIMULATED. Each row reconciles: transfers = Lyft completed + Lyft no-show + cancelled in the lag + cancelled after booking. Re-derived from `per_night_metrics.csv`.

Rule	Transfers	Lyft completed	Lyft no-show	Cancelled in lag	Cancelled after booking
R1(10)	196	94	7	18	77
R1(15)	104	55	4	11	34
R1u(10)	8	4	0	1	3
R2(15)	294	187	14	14	79
R3(2)	330	211	15	16	88

Five mechanisms (verified event behaviour; their size depends on the assumptions):

1. **A transfer does not stop the patience clock.** The lag and the Lyft delay come on top of the time already waited. Under R1(10), 95 of 196 transferred riders cancelled before their Lyft arrived; under R2(15) and R3(2), about a third did.
   Interpretation (corrected in methodology v1.6). A transfer does not end waiting; riders may still cancel before Lyft arrives. Riders moved by R2(15) and R3(2) had waited less when transferred (median 0 s, against 629 s under R1(10) and 932 s under R1(15)). Under the placeholder patience, where giving up becomes likelier with time waited, that is consistent with their lower *share* lost after transfer: a conditional calculation on those same riders and delays gives 44–47% against 29–30%, matching the observed shares. It is a possible contributing mechanism, not an isolated cause; the rules also differ in which riders they move. Under constant-hazard patience, elapsed waiting alone does not change the probability of surviving an equal additional waiting period. That does *not* establish that the cancellation gap between entire policies disappears, because a different patience model would also change who is transferred, when, and who survives in the vans. Whether the policy gap survives other patience shapes is untested.
2. **Assignment is not pickup.** R1u(10) fires only when a request has had no van in the plan for 10 minutes. On L4 it never fired after the vans came on duty. On L5 it fired 8 times on 6 of the 30 nights, all for requests made during the burst (18:21–18:28), with transfers at 18:35–18:38.
3. **Guardrail excesses happen between epochs.** There were 36 excess events across the 180 runs, from 16 distinct requests on 10 nights; the same request repeats under rules that made the same decisions. No plan ever created a violation. The attribution check finds that each one coincided with a van no-show on the same van while the rider was aboard, with an excess of at most 150 s per no-show plus a 13-s matrix tolerance. This is *consistent with* the no-show wait causing the delay; it is not a proof. None was unexplained.
4. **Denominators matter.** Wait percentiles cover only requests picked up: on average 16.1 of 28 under R0 and 20.2 under R2(15). The share of *all* requests reached by a vehicle within 15 minutes (curb arrival, not boarding) is therefore reported alongside.
5. **The 15-minute measure interacts with transfer timing** (added in methodology v1.6). A request moved under R1(10) has already waited at least 10 min, then faces the 2-min lag and a Lyft delay with median 8 min, so it reaches the curb within 15 min only if the delay is under

about 3 min (roughly 2%); under R1(15), never. The waiting-time rules can therefore gain on this measure only by freeing vans, while R2(15) uses the same 15-min threshold as the measure. Measures without a threshold give the same ordering: requests picked up (20.2–20.4 under R2(15) and R3(2), against 16.1–17.2) and the 90th-percentile wait (Table 19). Reporting at 10 and 20 min (spec §4) has not yet been run.

Table 19: Descriptive measures on L5: mean of the 30 nightly values, with the standard deviation across nights in brackets DESIGNED CHECK SIMULATED. They describe behaviour; they do not rank the rules. Re-derived from `per_night_metrics.csv`.

Per night	R0	R1(10)	R1(15)	R1u(10)	R2(15)	R3(2)
Reached in 15 min, share of all 28	0.48 (0.09)	0.51 (0.08)	0.50 (0.09)	0.48 (0.10)	0.73 (0.07)	0.75 (0.08)
Wait P90, picked up (min)	20.2 (5.3)	18.1 (2.4)	19.6 (4.0)	20.2 (4.6)	13.3 (1.5)	12.2 (1.7)
Lyft bookings	0.0	5.9	3.1	0.2	9.3	10.5
Van-km	51.2	47.6	48.6	51.9	45.7	43.0

7.8 Reading the results as trade-offs

Two objectives are stated explicitly: service S, the share of all requests reached within 15 minutes (larger is better), and Lyft use B, the bookings per night (smaller is better). Rule P *dominates* rule Q if $S_P \geq S_Q$ and $B_P \leq B_Q$, with at least one inequality strict. The calculation uses saved outputs only.

- **On the 30-night means,** R0, R1(15), R1(10), R2(15) and R3(2) are nondominated among the six tested rules (Fig. 9). R1u(10) is dominated by R0, narrowly: $S = 0.476$ against 0.477 (about one request in 840), with more Lyft use. No noise margin is applied (D-50), so in practice it is a near tie that uses slightly more Lyft. R0 is nondominated through zero Lyft use alone. The same five remain nondominated with van-km as a third objective.
- **Night by night,** R2(15) and R3(2) reach more requests than R0 on all 30 nights; R1(10) reaches more on 19 nights and fewer on 6.

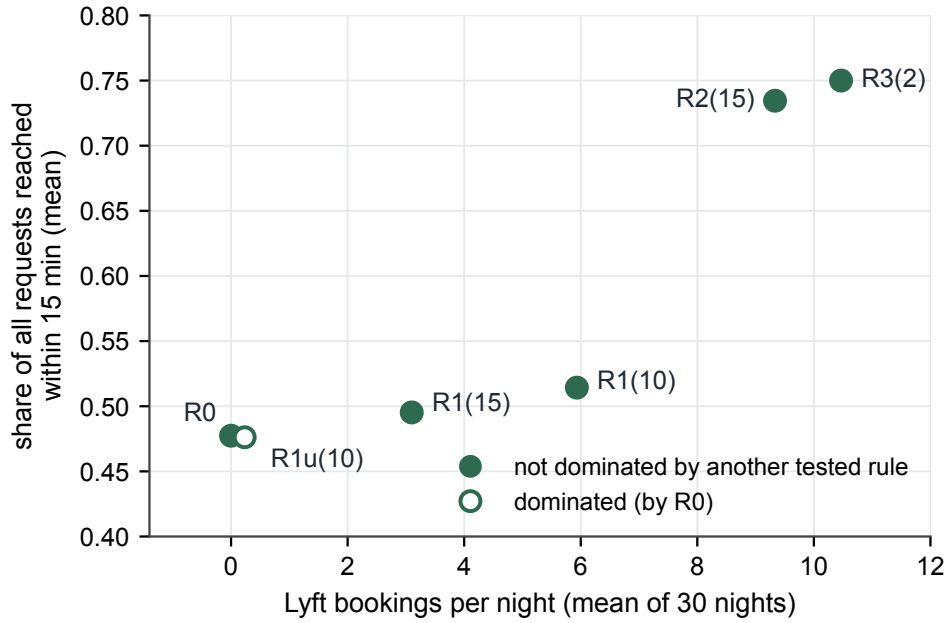


Figure 9: Lyft use against service on L5, 30-night means for the six tested rules DESIGNED CHECK SIMULATED. Filled points are not dominated by another tested rule; R1u(10) is dominated by R0. Five tested points are *not* a frontier of all policies and not a recommendation. Drawn from `tradeoff_rule_summary.csv`.

Three trade-offs are kept separate:

1. the fixed weights inside C's routing objective (Section 5);
2. the empirical trade-off between transfer rules (this analysis);
3. the decision criteria for choosing a rule (D-50, not approved).

"Nondominated among six tested rules" is neither a frontier of all policies nor a recommendation. Because the service measure favours early transfer (mechanism 5 in Section 7.7), another measure or decision criterion could change the assessment. Still to come are paired intervals, once D-50 is decided; threshold sweeps to trace curves; calibrated behaviour; and five vans.

7.9 A leader–follower reading (possible extension, not implemented)

One possible extension is to optimize transfer policy while anticipating how the routing dispatcher responds. A bilevel formulation is a modelling choice: a threshold could also be tuned by simulation-based search with the dispatcher treated as a black box. One reading has two decision makers. The *leader* is the policy designer, who chooses rule parameters θ. The *follower* is the routing optimizer, which responds to the requests left after transfers. As a bilevel program:

$$\min_{\theta} \ \mathbb{E}\big[L(\theta, y) + \lambda\, B(\theta)\big] \quad \text{subject to} \quad y \in \arg\min_{y' \in Y(\theta)} f(y'; \theta). \tag{6}$$

What we implement is a sequential policy simulation. In each epoch the dispatcher plans, the rule reads the plan, transfers are applied and the dispatcher re-plans once; six fixed rules are compared by replaying nights.

What a finite search would and would not show (corrected in methodology v1.6). An exhaustive search over a defined finite set of θ values, with the follower defined as our deterministic per-epoch restricted MILP and its tie rule, would identify the best option within that set for the stated objective averaged over the sampled nights. It would not establish optimality over continuous thresholds, over all policies, or for the true expectation.

To make the formulation precise we would still need to define:

- the leader's decision space (rule family; discrete or continuous θ; bounds);
- the follower: a sequence of per-epoch restricted MILPs whose state depends on earlier epochs and random outcomes, or one whole-evening problem, with its information and timing;
- how non-unique follower optima are treated (optimistic or pessimistic; ours uses a fixed canonical tie rule, D-52, which is neither);
- the expectation (a scenario distribution, or a sample average over N nights);
- the loss L and the weight λ (D-50 open);
- any upper-level constraints, such as a Lyft budget or a service floor.

Any single-level reduction would need specific assumptions and justification, and **none is claimed**. Riders (fixed patience rules) and Lyft (a delay distribution) are not modelled as optimizing players. The terminology follows von Stackelberg (1934), Colson, Marcotte and Savard (2007), and Dempe (2002).

7.10 Status by item

Table 20: Status by item after M0–M3's batch (`scenario_methodology.md` v1.6 §7).

Status	Items
Implemented and verified	Event engine for L1–L4 events (requests, movement, boarding, drop-offs, curb waits and no-shows, designed cancellations, commitment, periodic re-planning, backlog completion); dispatcher C with pending price A (approved provisionally); the curb-wait convention D-54; the Lyft rules all evening on L4 with the full transfer lifecycle (M2); logs, narratives and reports
Implemented, provisional	All D-52 settings ($K = 3$, $\Psi = 1000$, solver budget, tie rule); presence $p = 0.089$ (L1–L4 do not depend on it); the conventions of Section 4.10
Implemented (M3 pilot and batch)	L5 pilot settings in the run configuration (D-56); resumable 30-night batch with per-run checks (D-57); descriptive trade-off analysis of saved outputs
Documented only	Decision criteria and paired intervals (D-50, not approved); pre-registration
Not started	Heuristic T; hindsight H; the L6 and L7 builders; calibration; the M5 benchmark; the experiment grid

7.11 Limitations to keep in view

- C is optimal **over its candidate set at each epoch, subject to solver status**; it is not optimal over all routes or over the evening, and it is myopic.

- $K = 3$ is a tractability choice for pure Python. It shapes ETAs and pending status, so it is part of the held-fixed dispatcher.
- Guardrail excesses can still arise between epochs (after a no-show wait, or where the travel matrix breaks the triangle inequality by up to 12.7 s). None occurred in the L1–L4 runs. In the L5 batch there were 36 excess events (16 distinct requests on 10 nights), all attributed to van no-shows by the batch check (Section 7.7). Reports must attribute any that occur.
- During an absent rider's curb wait the planner keeps the pickup committed and plans for a possible boarding at expiry (D-54, adopted 22 October; planning only, not Hopkins policy).
- Shift ends (tapering, L7) are not implemented, and the rule's wording is open.
- The off-duty trigger differs between the specification and the code (Section 4.7); recorded, not changed.

7.12 Next milestone and how the work grows

M3: batch done, one step remaining. The L5 pilot (D-56) and the 30-night batch (D-57) are done (Section 7.7). The remaining step is the paired-interval analysis of the *saved* outputs, which needs no new simulation. Its entry decision is D-50: which primary metric, which margins and which power target. That analysis will show the interval machinery working and the night-to-night variance the power check needs. With two vans and placeholder behaviour, it supports no conclusion about the real service.

Table 21: Milestones (`scenario_methodology.md` v1.6 §8). M0–M2 and the M3 batch are done; the rest is a plan.

Milestone	Adds	Shows	State
M0–M1	Referee; C on L4 under R0	Exact answer keys; joint optimization on overlapping requests	Done, verified
M2	The Lyft rules all evening with C on L4	Transfer bookkeeping through a full evening	Done, verified
M3	L5, 30 nights × 6 rules	Pairing, metrics, variability; paired intervals after D-50	Batch done; intervals wait on D-50
M4	The L6 builder, 64 / 50 / 54 (needs go-ahead)	Inputs for the demonstration	Planned
M5	First hour: 5 vans, 64 requests; benchmark	Whether C fits the budget, confirming or overturning D-51	Planned
M6	The full 168-request, five-van evening	One traced evening, end to end	Planned
M7	The experiment grid	The research answer	Planned

7.13 Open decisions for the team

1. D-50: the decision criteria (primary metric, margins, power target), before paired intervals are computed.
2. The midterm timing is settled: a 30-minute slot, with the presentation planned at about 27 minutes.
3. Review of the provisional D-52 settings ($K = 3$, Ψ, solver budget, tie rule) and of the proposed conventions D-46, D-47(2), D-48 and D-50.
4. Later, not now: accepting the dashboard status mapping that pins $p \approx 8.9\%$, and the calibration targets.

Settled on 22 October: the pending price (A, provisionally, D-53 approval), the curb-wait convention (D-54) and the L5 pilot settings (D-56).

Table 22: Items that are provisional, proposed or open in the source documents.

Item	Current value or state	Status
Held-fixed van dispatcher (D-51)	C, conditional on the M5 benchmark	provisional
Simulator build conventions (D-46)	spec §17	proposed, not approved
Guardrail convention; solver settings (D-47(2), (4))	no new or increased violation; gap 0, node limit	proposed, not approved
Candidate rule (D-52 and amendment)	$K = 3$ new pickups per route; all stops reordered	provisional
Pending-price formulation (D-53)	A, the default; original and B kept as options	provisional
Curb-wait convention (D-54)	pickup stays committed; possible boarding at expiry, planning only	provisional
L5 pilot settings (D-56)	lag 2 min, consent 1.0, $p = 0.089$, in the run configuration	provisional
Decision criteria (D-50)	primary metric, margins, power target	open
No-show probability p (D-48)	$\approx 8.9\%$ per arrival under the mapping	proposed, not approved
Margins and primary metric (D-50)	1 min / 2 points; share not reached in 15 min	proposed, not approved
Request count (D-35, D-36)	64 / 50 / 54 = 168	provisional
Patience distribution and range (D-39)	Weibull 1.5, median 18; 50–75%	provisional
L6 base geography (D-34)	reference or dispersed	provisional
Status mapping behind p	not accepted	open
Shift-end rule (L7 only)	two wordings; not implemented	open

A Decision history

A summary of the decision-log rows that shape the current scenario and simulator, with their recorded status. The decision log is the authority; this table abbreviates it.

Table 23: Decision history (abbreviated from `decision_log.md`).

ID	Date	Decision	Status
D-01	29 Sep	The origin and destination maps count completed rides	Derived
D-02	29 Sep	No replay of April; build an April-shaped base scenario	Team decision
D-03	29 Sep	Rides after midnight belong to the previous service night	Assumed
D-08	29 Sep	Party size mean 1.39, shape assumed	Mean derived, shape assumed; base shape now D-38
D-16	1 Oct	Requests start and end at bubble points	Design choice
D-17	1 Oct	Depot: heaviest origin bubble on the Homewood campus	Assumed
D-18	1 Oct	Gravity × IPF pairing, $\beta_{\text{geo}} = 0.1$/min	Design choice; β_{geo} assumed
D-20	1 Oct	v1 request counts and arrivals	Paused (12 Oct)
D-21	1 Oct	Exponential patience, median 12 min	Under revision (12 Oct); see D-39
D-22	1 Oct	Van capacity 4	Under revision (12 Oct); 14 seats in D-30
D-23	12 Oct	Homewood only; no East Baltimore	Accepted
D-24	12 Oct	Door-to-door; hotspots are not stops	Accepted
D-25	12 Oct	Direction: a TransLoc-like re-planning dispatcher is the fixed baseline	Accepted (direction)
D-26	12 Oct	Primary question: Lyft transfer rules, dispatcher held fixed	Accepted (direction)
D-27	20 Oct	Homewood including Peabody	Accepted
D-28	20 Oct	Keep all Lyft rules; run one at a time; explain the winner	Accepted
D-29	20 Oct	Weighted objective: convex φ, $\beta = 0.5$, $\gamma = 0.1$, guardrail form, reporting thresholds	Accepted
D-30	20 Oct	5–6 vans with 14 seats; demand scaled to the fleet	Accepted
D-31	20 Oct	Local OSRM travel matrix and manifest	Accepted
D-32	20 Oct	Level ladder L1–L7	Accepted

ID	Date	Decision	Status
D-33	21 Oct	Parties of five can use Lyft XL, kept together	Accepted modelling instruction (not verified policy)
D-34	21 Oct	Dispersed-geography variant	Implemented and verified; L6 base provisional
D-35	21 Oct	Volume formula $N_h = \rho_h v_h$, largest remainder	Provisional
D-36	21 Oct	Five constant vans; 64/50/54 = 168; backlog completion	Accepted; count provisional
D-37	21 Oct	Sixth van on the same requests	Accepted
D-38	21 Oct	Average-April party-size shape	Accepted
D-39	21 Oct	Patience as a personal deadline; Weibull placeholder; 50–75% range	Mechanism accepted; values provisional
D-40	21 Oct	No-show checked at curb arrival; 180-s wait	Mechanism accepted; values provisional
D-41	21 Oct	Shared presence draw	Accepted design; simplified by D-48
D-42	21 Oct	Silent abandonment	Sensitivity only
D-43	21 Oct	Lightweight Lyft model; three failure paths; no rebooking	Accepted design; values provisional
D-44	21 Oct	Lyft cost ledger	Financial part superseded by D-45
D-45	21 Oct	No money modelled; operational counts only	Accepted
D-46	21 Oct	Simulator build conventions (spec §17)	Proposed, not separately approved
D-47	21 Oct	Model-math corrections: (1) multipliers once, (3) pending claim, (5) wording; (2) guardrail convention, (4) solver settings	(1), (3), (5) corrections; (2), (4) proposed
D-48	21 Oct	One presence probability for all modes; $p \approx 8.9\%$ under the mapping; errata for 0.047 fields	p proposed, not approved; errata corrections
D-49	21 Oct	Evidence-wording corrections (targets, range, 29 days, pairing)	Corrections
D-50	21 Oct	Pre-registration statistics	“CI contains 0” fix a correction; margins, metric, power proposed
D-51	21 Oct	Held-fixed dispatcher: C provisionally; T a contingency, not a runtime fallback	Provisionally decided

ID	Date	Decision	Status
D-52	22 Oct	M0–M1 implementation settings: $K = 3$ candidate rule, cost units, solver settings, engine conventions	Implemented; provisional
D-52 (am.)	22 Oct	Candidate rule: all of the van's stops reordered freely	Implemented; provisional (team review pending)
D-53	22 Oct	Pending price can defer the longest waiters	Open: proposal, decision needed
D-52 (am. 2)	22 Oct	Fixes from the independent math review (exact integer objective, wall-clock limit stops the run, numeric van order in ties)	Implemented
D-52 (am. 3)	22 Oct	Status handling and plan audit: 'empty' label; solver error or infeasibility stops the run; incumbent at the node budget executed as feasible, unproven; per-epoch audit stops on any violation	Implemented
D-53 (upd.)	22 Oct	Review findings and proposals A and B for the pending price	Open: team decision needed
D-53 (upd. 2)	22 Oct	Pending-price experiment: three selectable formulations compared on L4; recommendation A	Experiment done; A awaits team approval; default stays original
D-54	22 Oct	Curb-wait planning: pickup stays committed; possible boarding at expiry for planning only; obligation removed at no-show expiry	Adopted as planning default; provisional
D-52 (am. 4)	22 Oct	Robustness fixes: exact-integer tie enumeration; plans voided once all requests are resolved; richer epoch logs	Implemented
D-53 (appr.)	22 Oct	Pending price A approved after a separate experiment approval; A becomes the default; original and B kept as options	Approved, provisional
D-55	22 Oct	M2: Lyft rules with C over the whole L4 evening; rule pass, what a rule reads, R1u clock and consent made explicit	Implemented and verified
D-52 (am. 5)	22 Oct	Tie-search robustness: a failed tie re-solve is labelled incomplete instead of stopping the run	Implemented
D-56	22 Oct	L5 engineering pilot (two runs); L5 settings in the run configuration	Done; settings provisional
D-57	22 Oct	L5 30-night batch (180 runs): pipeline verification and variability; descriptive only	Done

Proposals still open from 12 October include P-05 (rolling-horizon details), P-08 (patience form, superseded in part by D-39), P-11 (HiGHS with a lock file) and P-12 (request-hour reading, now a transparent base assumption under D-49).

B Symbols and units

Table 24: Symbols and units.

Symbol	Meaning	Unit
t_r	Request time of booking r	s from 18:00
q_r	Passengers in booking r	passengers
W_r	Wait: pickup instant (curb arrival of the vehicle the rider boards, either mode) minus request time	min
H_r	Ride time: drop-off arrival minus pickup instant (includes the 30-s boarding dwell)	min
T_r	Direct van time $t_0(o_r, d_r)\, m_v$	min
$t_0(i, j)$	Base matrix travel time, unscaled	s
m_v, m_L	Van and Lyft travel multipliers, each applied once	–
$\varphi(W)$	Convex waiting penalty	points per passenger
β	Ride weight (0.5)	points per passenger-min
γ	Distance weight (0.1)	points per km
G_r	Ride guardrail, $\max(25, 2T_r + 5)$	min
Ψ	Penalty for an unavoidable guardrail violation	points per passenger-min
π_r, M	Pending price; large pending constant	points
b_r	Pending price excluding M, by formulation (original, A or B)	points
L_r	Earliest physically possible pickup of r by any van (formulation A)	s
t_{next}	Time of the next epoch tick	s
y_{vk}, u_r	Van v uses route k; request r left pending	binary
S_k, C_v	Open requests added by route k; van v's committed pickup	sets
ρ_h	Requests per available van in hour h	requests per van-hour
C_h, V_h, v_h	April completed rides in hour h; April average vehicles; scenario vans	rides; vehicles
N_h	Scenario requests in hour h	requests
β_{geo}	Distance decay in the O–D model (0.1)	per minute
U_r, p	Presence draw; presence threshold (absent if $U_r < p$)	–
p_c	Consent probability (base 1)	–
ℓ	Dispatcher lag from decision to booking (2)	min
λ_r	Hidden Lyft delay from booking to arrival	min
θ	Wait threshold of rule R1	min
κ	Congestion threshold of rule R4	pending per van
τ	Current epoch time	s

Symbol	Meaning	Unit
Δ	Periodic re-planning tick (120)	s
g	Shift-end grace (0)	s
K	New pickups allowed per candidate route (3 in M0–M1, D-52)	requests

C Points where the sources disagreed, and how they were settled

These points were noticed while the draft of this companion was prepared (21–22 October). This document did not resolve any of them; the authors of the authoritative documents did, on 22 October, in the places named below. Items still open are marked as such. The same list, with exact locations, is kept in `methodology_pdf/CONFLICTS.md`.

Table 25: Disagreements found in the sources and where they were settled.

Point	How it was settled	Where
D-52 referenced before it existed	D-52, its two amendments and D-53 added	decision log
Which dispatcher is held fixed	C provisionally held fixed; T a later robustness variant or contingency, never a runtime fallback; H4 now compares C with T	spec v0.5 §4, §9.2; model design v0.3 §6.1; pre-registration §1–2, H4; methodology v1.1 §8
Implementation status	Verified M0–M1 status added (Section 7)	methodology v1.1 §7
Co-primary P90 wait	P90 wait among picked-up riders is secondary, consistent with D-50	pre-registration §3
Shift-end rule	Marked **open**: two wordings, L7 only, not implemented	spec §3.1, §17.6; model design §3.3
Stale design-trail text	Trail §1, §2 and §8 annotated in place	decision log
Candidate cap wording	"Relative to the previous plan" superseded by D-52	decision log D-47(3); model design §3.3
Epoch triggers	Only cancellations and no-shows of requests in van planning trigger; Lyft-side events never do	spec §17.4
Can a committed rider cancel?	Yes: patience checks include `committed` (the L3 key relies on this)	spec §17.3
Which instant is "pickup"?	Curb arrival of the vehicle the rider boards; the dwell follows and is part of the ride	spec §17.5
L4 rule set	R0 plus five active cases; 22 transfers and 4 declines are totals across rule runs	methodology v1.1 §7
Off-duty trigger	**Open:** spec §17.4 and D-46(3) list it as a trigger; the code logs it and opens no epoch. Recorded, simulator unchanged	spec §17.4; `src/sim/engine.py`; methodology v1.6 §7b

Wording residues found in the final check, and settled. Six smaller residues were then fixed in place: the spec §17 heading now says implemented for L1–L4 in M0–M1; the spec §8 L4 row lists R0, R1(10), R1(15), R1u(10), R2(15) and R3(2); the `model_design.md` status line and §6.1 now match D-51; its §5 trigger row is narrowed to match spec §17.4; design trail §8 is annotated (under the mapping, $p \approx 8.9\%$ follows regardless of patience); and the pre-registration title reads revised 22 Oct".

A last residue, also settled. `scenario_methodology.md` v1.1 §8 had said that "a failed solve executes committed work only" without the other cases. It now matches D-52 amendments 2–3: a solve with no plan executes committed work only; one reaching the node budget executes its best plan, labelled feasible, unproven; a solver error, an infeasible model or the wall-clock safety stop ends the run with an error.

D Source documents and versions

This companion was prepared from the files below, last read on 23 October 2026. The SHA-256 fingerprints identify the exact versions; the full values are in `methodology_pdf/SOURCES.md`.

Table 26: Source documents.

File	Version	SHA-256 (first 16 hex)
`scenario_methodology.md`	v1.6, 23 Oct 2026	`bd93a4548c987d12`
`scenario_spec.md`	v0.8, 22 Oct 2026	`0cdb285245508c1a`
`model_design.md`	v0.5 draft, 22 Oct 2026	`407d2d5bcfb7f1c5`
`decision_log.md`	through D-57, 22 Oct 2026	`653f09658cf34501`
`preregistration_v2.md`	draft, revised 22 Oct 2026	`6b05cf0ffed51b73`
`outputs/sim/L4_R0_M1_reference/epochs.jsonl`	M1 run, original price (Table 14), read only	`7b962cc5a96ee9e0`
`outputs/sim/d53_experiment/d53_comparison.md`	D-53 experiment (Table 15), read only	`38f11a09deafbc91`
`outputs/sim/m2_L4_report.md`	M2 report (Tables 16 and 17), read only	`dbc1e1dc841d5cf2`
`outputs/sim/batch_L5_v1/per_night_metrics.csv`	L5 batch (Tables 18 and 19), read only	`1e26c95e1c92defd`
`outputs/sim/batch_L5_v1/tradeoff_rule_summary.csv`	Fig. 9, read only	`e94a8c0f7588c798`

Not used as authority: the paused v1 `scenario_contract.md`, the archived versions in `archive/`, and the restricted evidence folder. April numbers are the aggregates already quoted in the files above.

E References

Literature and vendor sources are cited as the source documents cite them; full bibliographic details and access levels are in `research_notes/math_literature.md` and `transloc_operations.md` §12.

Project documents

- Team FourSight. *Scenario-design methodology and build contract*, v1.6, 23 Oct 2026 (`scenario_methodology.md`).
- Team FourSight. *Scenario specification*, v0.8, 22 Oct 2026 (`scenario_spec.md`), especially §17, the simulator build contract.
- Team FourSight. *Mathematical design and architecture*, v0.5 draft, 22 Oct 2026 (`model_design.md`).
- Team FourSight. *Decision log* (plan v2), D-01 to D-57 with the scenario design trail (`decision_log.md`).
- Team FourSight. *Pre-registration v2*, draft revised 22 Oct 2026 (`preregistration_v2.md`).

Literature

- Alonso-Mora, J., Samaranayake, S., Wallar, A., Frazzoli, E. and Rus, D. (2017). On-demand high-capacity ride-sharing via dynamic trip-vehicle assignment. *PNAS* 114(3), 462–467.
- Berbeglia, G., Cordeau, J.-F. and Laporte, G. (2010). Dynamic pickup and delivery problems. *European Journal of Operational Research* 202(1), 8–15. (Cited as the class survey; abstract-level access.)
- Colson, B., Marcotte, P. and Savard, G. (2007). An overview of bilevel optimization. *Annals of Operations Research* 153(1), 235–256. doi:10.1007/s10479-007-0176-2.
- Dempe, S. (2002). *Foundations of Bilevel Programming*. Nonconvex Optimization and Its Applications, vol. 61. Kluwer. doi:10.1007/b101970.
- Grahn, R., Qian, S. and Hendrickson, C. (2023). Optimizing first- and last-mile public transit services leveraging TNCs. *Transportation* 50(5), 2049–2076.
- Ho, S.C., Szeto, W.Y., Kuo, Y.-H., Leung, J.M.Y., Petering, M. and Tou, T.W.H. (2018). A survey of dial-a-ride problems: literature review and recent developments. *Transportation Research Part B* 111, 395–421. (Abstract-level access.)
- Jaw, J.-J., Odoni, A., Psaraftis, H. and Wilson, N. (1986). A heuristic algorithm for the multi-vehicle advance request dial-a-ride problem with time windows. *Transportation Research Part B* 20(3), 243–257. (Abstract-level access.)
- von Stackelberg, H. (1934). *Marktform und Gleichgewicht*. Springer, Vienna.

Vendor, institutional and software sources

- TransLoc TechLog. *Ridesharing Algorithms in TransLoc OnDemand* (2016 post; archived copy, April 2018). Historical description of the engine; the current version is unknown.

- Johns Hopkins Transportation. *Shuttle Services / Night Ride* pages: archived captures (2023–2026) and the live page read on 21 Oct 2026 (targets; van and Lyft wait up to 3 minutes; dispatcher-sent Lyft).
- The Johns Hopkins News-Letter. *Lyft partnership expands Blue Jay Shuttle services*, Feb 2018 (the earlier "5+ riders get a van" statement; superseded for modelling by D-33).
- Lyft. Driver help on no-shows (recorded after arrival and the wait timer) and general Concierge guidance (undated page, read 21 Oct 2026), as cited in D-43 and design trail §9.
- Project OSRM, v6.0.0, with the Geofabrik Maryland extract (local server; matrix fingerprints in `MANIFEST.json`).
- HiGHS linear and mixed-integer optimization solver (MIT licence), as cited in the methodology and `research_notes/math_literature.md`.